PROJET IOT

Système IoT de Monitoring de la Sécurité Réseau en Environnement Hospitalier IoMT

Réaliser par :

Saad Sainane

Khalid Nait Ali

Zakaria Ftissa

El Mehdi Amar

Encadrer par :

Mme.Bouhaddou

Année universitaire : 2025/2026

Resume

Ce projet porte sur la conception et l'implémentation d'un système IoT de monitoring de la sécurité réseau dans un environnement hospitalier connecté. L'objectif principal est de surveiller le trafic réseau des dispositifs IoMT afin de détecter les comportements anormaux pouvant indiquer une intrusion, une attaque ou une activité malveillante susceptible de compromettre la disponibilité et la sécurité des services médicaux.

Le système proposé repose sur une architecture IoT en quatre couches comprenant la collecte des données réseau (Sensing Layer), la transmission des informations selon différentes stratégies (Network Layer), le traitement intelligent des données à l'aide de modèles de Machine Learning légers (Data Processing Layer) et la visualisation des résultats à travers un tableau de bord local (Application Layer). Les données utilisées dans ce projet sont issues d'un dataset public spécialisé dans le trafic réseau IoMT, permettant de simuler différents scénarios de trafic normal et d'attaques.

Plusieurs stratégies de transmission des données ont été comparées afin d'évaluer leur impact sur la latence, la bande passante et la consommation énergétique. De plus, différents modèles de détection d'anomalies ont été entraînés sur PC puis déployés sur Raspberry Pi afin d'illustrer l'approche Edge Computing et de respecter les contraintes de ressources d'un système embarqué.

Les résultats montrent que les modèles légers, notamment les arbres de décision, permettent d'obtenir une bonne capacité de détection tout en conservant un temps d'inférence réduit et une faible taille de modèle. Le système développé permet également de suivre les prédictions, les alertes, les métriques réseau et l'estimation énergétique à travers un dashboard local. Cette solution montre l'intérêt d'une approche Edge pour renforcer la sécurité des réseaux IoMT hospitaliers.

Mots-clés : Internet of Things, IoMT, Raspberry Pi, Edge Computing, sécurité réseau, détection d'intrusions, Machine Learning, monitoring hospitalier.

Remerciements

Nous adressons nos sincères remerciements à Mme Bouhaddou pour son encadrement, sa disponibilité, ses remarques constructives et ses orientations techniques tout au long de la réalisation de ce projet. Ses conseils nous ont permis de mieux structurer notre travail, d'améliorer nos choix techniques et de prendre en compte les contraintes réelles liées au déploiement d'un système IoT sur Raspberry Pi.

Nous remercions également l'équipe pédagogique de HESTIM pour la qualité de la formation dispensée et pour les connaissances théoriques et pratiques acquises durant notre parcours. Ces acquis nous ont permis d'aborder ce projet avec une meilleure compréhension des domaines de l'IoT, de l'Edge Computing, de la sécurité réseau et de l'intelligence artificielle appliquée.

Nous tenons aussi à remercier nos collègues pour leur collaboration, leur esprit d'équipe et les échanges enrichissants qui ont contribué à l'avancement du projet. Leur soutien et leurs remarques nous ont aidés à surmonter certaines difficultés techniques rencontrées pendant les phases d'entraînement, de déploiement et de test.

Enfin, nous exprimons notre gratitude à toutes les personnes qui nous ont encouragés et soutenus, directement ou indirectement, durant la réalisation de ce travail. Leur aide et leurs encouragements ont constitué une source de motivation importante pour mener ce projet à terme.

Table des matières

Chapitre 1 : Contexte et problématique 7

1.1 Contexte du projet : 7

1.2 Problématique : 7

1.3 Objectifs du projet : 8

1.4 Méthodologie et processus de travail : 9

1.5 Conclusion : 10

Chapitre 2 : Dataset, sélection du dataset et benchmark 11

2.1 Choix du dataset : 11

2.2 Description et selection : 11

2.2.1 Visualisation du dataset et distribution des classes : 12

2.3 Benchmark : 14

2.4 Conclusion : 15

Chapitre 3 : Entrainement, Energie, modèles et amélioration : 16

3.1 Préparation des données : 16

3.2 Entrainement des modèles : 16

3.2.1 Environnement logiciel et système d'exploitation : 17

3.3 Résultats des modèles : 20

3.4 Mesure énergétique logicielle : 27

3.5 Améliorations : 27

3.6 Conclusion : 31

Chapitre 4 : Inférence, Raspberry Pi, résultats des mesures et Dashboard 32

4.1 Compression locale avant déploiement : 32

4.2 Déploiement Raspberry Pi : 33

4.3 Sensing Layer sur Raspberry Pi : 34

4.4 Network Layer sur Raspberry Pi : 36

4.5 Data Processing Layer sur Raspberry Pi : 37

4.6 Dashboard : 39

4.7 Analyse finale : 45

4.8 Conclusion : 46

Conclusion générale : **46**

Liste des figures

Figure 1 : Doiagramme de Gannt 9
Figure 2 : Schéma processus de travail 10
Figure 3 : Extrait des premières features du dataset CICIoMT2024 12
Figure 4 : Distribution des classes dans l'échantillon stratifié CICIoMT2024 14
Figure 5 : Préparation du dataset pour la classification binaire 17
Figure 6 : Mesure logicielle du temps, CPU, RAM et énergie 18
Figure 7 : Modèles binaires exportés au format joblib 19
Figure 8 : Export des modèles multi-classes au format joblib 19
Figure 9 : Comparaison des performances des modèles selon l'accuracy 21
Figure 10 : Comparaison de l AUC des modèles 22
Figure 11 : Comparaison de la taille des modèles sérialisés 23
Figure 12 : Matrice de confusion Decision Tree binaire 24
Figure 13 : Courbe ROC macro-moyenne du modèle Decision Tree multi-classe 25
Figure 14 : F1-score par classe du modèle Decision Tree multi-classe 26
Figure 15 : Analyse du temps d'exécution des différentes étapes du pipeline de traitement 28
Figure 16 : Énergie consommée par étape du pipeline de traitement 29
Figure 17 :RAM par étape du pipeline 31
Figure 18 : Comparaison de la taille des modèles joblib et ONNX 32
Figure 19 : Installation et vérification des bibliothèques sur Raspberry Pi 34
Figure 20 : Structure des fichiers du projet sur Raspberry Pi 34
Figure 21 : Exécution de la couche Sensing Layer sur Raspberry Pi 34
Figure 22 : Exécution de la couche Network Layer sur Raspberry Pi 36
Figure 23 : Exécution de la couche Data Processing Layer sur Raspberry Pi 37
Figure 24 :Lancement du dashboard Flask 39
Figure 25 : Vue générale du dashboard IoMT 39
Figure 26 : Affichage des prédictions générées par le modèle 40
Figure 27 : Résumé de la couche Sensing Layer 41
Figure 28 : Résumé de la couche Sensing Layer 42
Figure 29 : Mesures de performance de la couche Data Processing Layer 43
Figure 30 : Comparaison des modèles de Machine Learning déployés 44

Liste des tableaux

Tableau 1 : Comparaison des performances et de la taille des modèles de classification 20
Tableau 2 : Résultats de la couche Sensing Layer sur Raspberry Pi 35
Tableau 3 : Comparaison des stratégies de transmission réseau 37
Tableau 4 : Analyse des performances du pipeline d'inférence 38

Introduction générale

Les systèmes hospitaliers modernes utilisent de plus en plus de dispositifs médicaux connectés afin d'améliorer la qualité des soins, d'assurer un suivi continu des patients et d'automatiser certaines tâches médicales. Ces équipements, regroupés sous le terme Internet of Medical Things (IoMT), peuvent inclure des capteurs biomédicaux, des moniteurs de signes vitaux, des pompes intelligentes, des dispositifs de suivi à distance ou encore des équipements médicaux intégrés au réseau hospitalier.

Cette évolution technologique présente de nombreux avantages pour les établissements de santé. Elle permet une meilleure réactivité du personnel médical, une collecte plus rapide des informations, une surveillance en temps réel et une optimisation de la gestion des équipements. Cependant, l'interconnexion croissante de ces dispositifs augmente également la surface d'attaque du réseau hospitalier. Un équipement médical connecté peut devenir une cible ou un point d'entrée pour des attaques réseau, ce qui peut compromettre la disponibilité des services, la confidentialité des données médicales ou la continuité des soins.

Dans ce contexte, la sécurité des réseaux IoMT devient un enjeu essentiel. Il est nécessaire de mettre en place des solutions capables de surveiller le trafic réseau, de détecter les comportements anormaux et de réagir rapidement aux intrusions. Toutefois, les systèmes IoT et IoMT fonctionnent souvent avec des ressources limitées en termes de mémoire, de puissance de calcul et d'énergie. Il est donc important de proposer une solution légère, rapide et adaptée à un environnement Edge.

Ce projet propose un système IoT de monitoring de la sécurité réseau dans un environnement hospitalier IoMT. Le système repose sur une architecture en quatre couches : la collecte des flux réseau, la transmission des données, le traitement intelligent par des modèles de Machine Learning et la visualisation des résultats à travers un dashboard local. L'objectif est de détecter les attaques et anomalies dans le trafic réseau tout en évaluant les contraintes de temps d'exécution, de mémoire, de CPU et de consommation énergétique.

Le travail suit une approche Edge Computing. Les modèles sont entraînés sur PC à partir du dataset CICIoMT2024, qui contient du trafic normal et plusieurs types d'attaques réseau. Afin de garantir des temps d'exécution raisonnables, un échantillon stratifié est utilisé pour conserver la distribution des classes. Les modèles entraînés sont ensuite exportés sous forme de fichiers sérialisés, puis déployés sur Raspberry Pi pour réaliser l'inférence et les mesures de performance.

Ce rapport présente les différentes étapes du projet : le contexte et la problématique, le choix du dataset, la préparation des données, le benchmark des modèles, les mesures énergétiques logicielles, le déploiement sur Raspberry Pi et l'analyse des résultats. Il met également en évidence l'intérêt d'une solution Edge pour renforcer la sécurité des réseaux IoMT tout en respectant les contraintes d'un dispositif embarqué.

Chapitre 1 : Contexte et problématique

1.1 Contexte du projet :

Les hôpitaux modernes s'appuient de plus en plus sur des dispositifs médicaux connectés afin d'améliorer la qualité des soins, d'automatiser certaines tâches médicales et d'assurer une surveillance continue des patients. Ces dispositifs, regroupés sous le terme **Internet of Medical Things (IoMT)**, incluent par exemple les moniteurs de signes vitaux, les pompes à perfusion intelligentes, les capteurs biomédicaux, les équipements d'imagerie médicale et les systèmes de suivi à distance. Grâce à ces technologies, les données médicales peuvent être collectées, transmises et analysées en temps réel. Cela permet aux professionnels de santé de réagir plus rapidement en cas de changement critique de l'état d'un patient, d'optimiser l'utilisation des équipements et de réduire certaines interventions manuelles. L'IoMT joue donc un rôle important dans la transformation numérique des établissements de santé.

Cependant, cette connectivité croissante introduit également de nouveaux risques de cybersécurité. Chaque équipement connecté devient un point d'entrée potentiel pour des attaques visant le réseau hospitalier. Des attaques comme le déni de service, le scan de ports, le spoofing ou l'injection de trafic malveillant peuvent perturber la communication entre les dispositifs médicaux, ralentir les services critiques ou compromettre la confidentialité des données de santé.

Dans ce contexte, la sécurité du réseau IoMT devient un enjeu essentiel. Il ne suffit plus de collecter les données médicales ; il faut également surveiller le trafic réseau afin de détecter rapidement les comportements anormaux. Le recours à une solution Edge basée sur Raspberry Pi permet d'effectuer une partie de cette détection localement, au plus près des données, tout en réduisant la latence et la quantité d'informations transmises vers un serveur central.

1.2 Problématique :

Le problème étudié dans ce projet est la détection locale d'intrusions et d'anomalies dans le trafic réseau IoMT d'un environnement hospitalier. Les dispositifs médicaux connectés échangent en permanence des données sensibles et doivent rester disponibles en continu. Une attaque réseau peut donc entraîner des perturbations importantes, comme l'indisponibilité d'un équipement, le ralentissement des communications ou l'exposition de données médicales confidentielles.

La difficulté principale consiste à détecter ces comportements suspects directement sur un équipement Edge, en l'occurrence un Raspberry Pi, sans dépendre entièrement d'un serveur central. Cette approche permet de réduire la latence et de réagir plus rapidement aux attaques, mais elle impose aussi des contraintes importantes en termes de temps d'exécution, d'utilisation mémoire, de charge CPU et de consommation énergétique.

Il est donc nécessaire de trouver un compromis entre la précision de détection des modèles, leur légèreté, leur rapidité d'inférence et leur coût énergétique afin de proposer une solution adaptée à un système IoMT embarqué.

1.3 Objectifs du projet :

L'objectif principal de ce projet est de concevoir et d'implémenter un système IoT de monitoring de la sécurité réseau dans un environnement hospitalier IoMT. Ce système vise à détecter les intrusions et les comportements anormaux dans le trafic réseau des dispositifs médicaux connectés, tout en optimisant l'utilisation des ressources d'un équipement Edge tel que le Raspberry Pi.

Plus précisément, ce projet vise à :

- **Surveiller le trafic réseau IoMT :**
  Observer les flux réseau générés par les dispositifs médicaux connectés afin d'analyser leur comportement et repérer les activités inhabituelles.
- **Détecter les attaques et anomalies :**
  Identifier les flux suspects pouvant correspondre à des attaques réseau, comme les DoS, DDoS, scans ou tentatives de spoofing.
- **Comparer des modèles ML légers :**
  Évaluer plusieurs modèles adaptés aux ressources limitées du Raspberry Pi, comme le thresholding, la régression logistique et l'arbre de décision.
- **Comparer trois stratégies de transmission :**
  Tester l'envoi immédiat, l'envoi par batch et l'envoi uniquement en cas d'anomalie afin de mesurer leur impact sur la latence, la bande passante et l'énergie.
- **Déployer l'inférence et le dashboard sur Raspberry Pi :**
  Exécuter les modèles entraînés sur le Raspberry Pi et afficher les résultats dans un tableau de bord local accessible depuis un navigateur.
- **Mesurer temps, CPU, RAM et énergie logicielle**
  Évaluer les ressources consommées par chaque étape du pipeline à l'aide de psutil, time.perf_counter() et d'une estimation énergétique basée sur 6.4 W.

L'ensemble du système est déployé sur un Raspberry Pi afin de reproduire les contraintes réelles des systèmes IoT embarqués et de valider l'approche Edge Computing dans un contexte de sécurité réseau hospitalière.

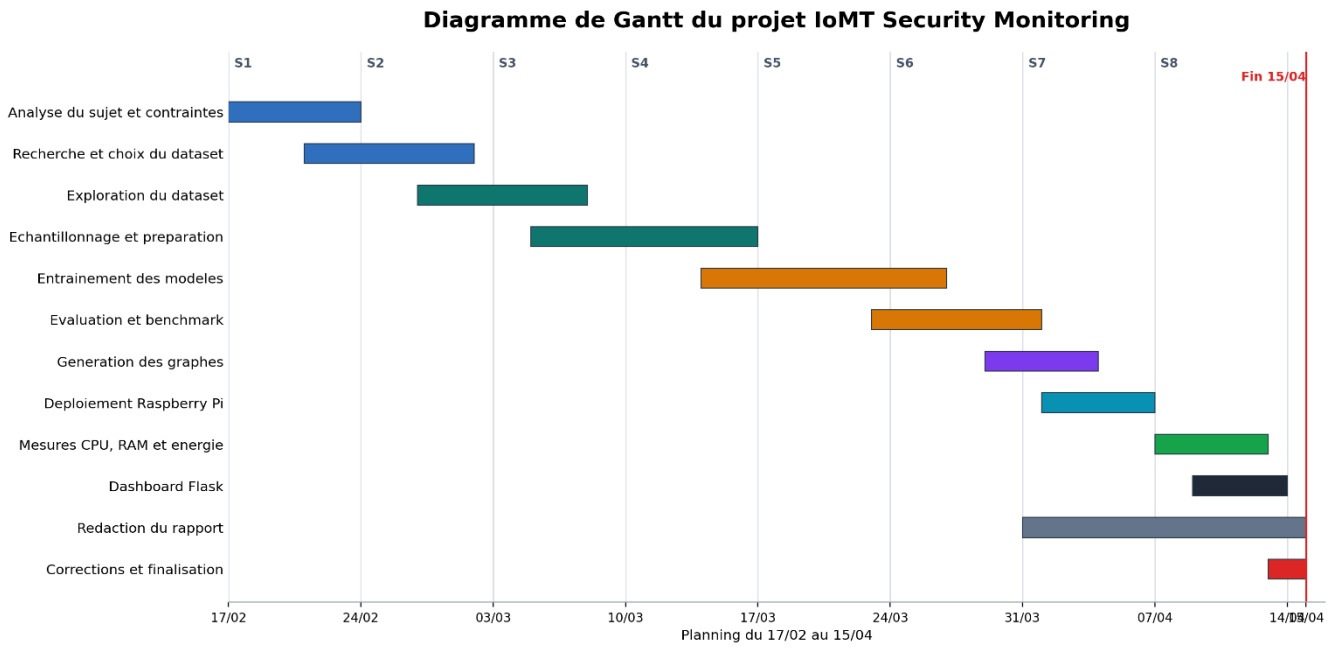
1.4 Méthodologie et processus de travail :

La méthodologie adoptée repose sur une organisation en plusieurs étapes complémentaires. La première étape consiste à analyser le sujet et à identifier les contraintes techniques imposées par un environnement IoMT hospitalier. Ensuite, le dataset CICIoMT2024 est sélectionné et préparé afin de produire un échantillon exploitable pour l'entraînement.

La deuxième étape concerne le benchmark des modèles Machine Learning. Plusieurs modèles légers sont comparés en classification binaire et multi-classe afin de choisir la solution la plus adaptée au déploiement Edge. La troisième étape consiste à exporter les modèles entraînés, à les transférer sur Raspberry Pi et à mesurer leur comportement réel en termes de temps, CPU, RAM et énergie logicielle.

Organisation des tâches : analyse du sujet, préparation du dataset, entraînement des modèles, génération des graphes, déploiement Raspberry Pi, mesure des ressources, développement du dashboard et rédaction du rapport.

Figure 1 : Doiagramme de Gannt



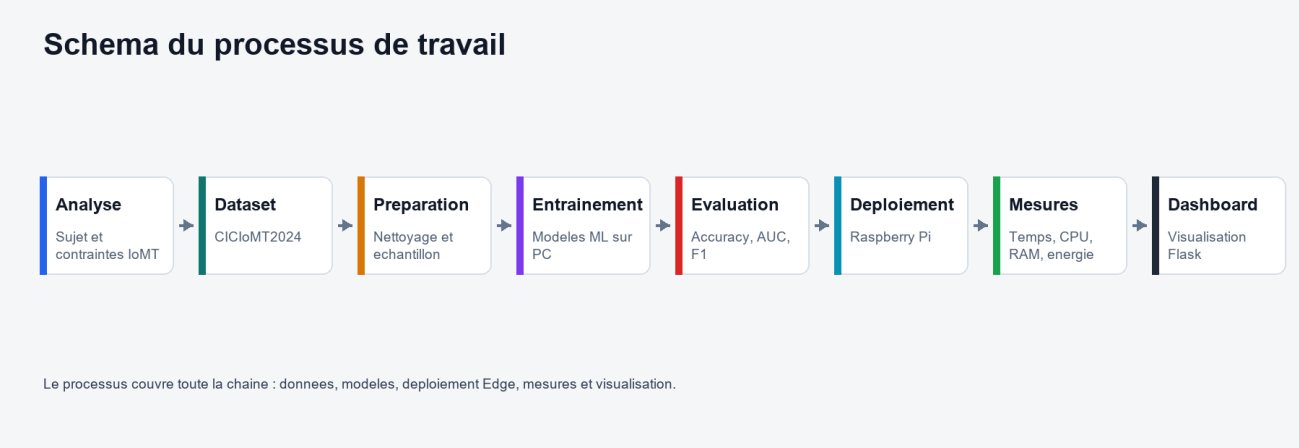
Ce diagramme présente la planification générale du projet entre le 17/02 et le 15/04. Il montre les principales étapes réalisées, depuis l'analyse du sujet jusqu'à la finalisation du rapport. Les tâches sont organisées de manière progressive, tout en gardant certains chevauchements entre les phases techniques et la rédaction.

Les premières étapes concernent l'analyse du sujet, le choix du dataset CICIoMT2024 et l'exploration des données. Ensuite, le projet se concentre sur la préparation de l'échantillon, l'entraînement des modèles Machine Learning et leur évaluation. La fin du planning est

consacrée au déploiement sur Raspberry Pi, aux mesures de performance, au développement du dashboard et aux corrections finales du rapport.

Ce diagramme montre que certaines activités ont été réalisées en parallèle, notamment la rédaction du rapport, les mesures sur Raspberry Pi et le développement du dashboard. Cette organisation a permis d'avancer efficacement tout en respectant la date de fin du projet.

Figure 2 : Schéma processus de travail



Ce schéma résume la méthodologie suivie pour réaliser le projet. Le processus commence par l'analyse du sujet et l'identification des contraintes liées à la sécurité réseau dans un environnement IoMT hospitalier. Ensuite, le dataset CICIoMT2024 est sélectionné comme source de données, car il contient du trafic normal et différents types d'attaques réseau.

Après la sélection du dataset, les données sont préparées à travers le nettoyage, la sélection des features et l'échantillonnage stratifié. Les modèles de Machine Learning sont ensuite entraînés sur PC afin d'éviter de surcharger le Raspberry Pi. Une phase d'évaluation permet de comparer les modèles selon plusieurs métriques comme l'accuracy, l'AUC, le F1-score et la taille du modèle.

Enfin, les modèles retenus sont exportés puis déployés sur Raspberry Pi pour effectuer l'inférence. Les mesures de temps, CPU, RAM et énergie logicielle sont collectées, puis les résultats sont affichés dans un dashboard Flask. Ce processus permet de couvrir toute la chaîne du projet, depuis les données jusqu'à la visualisation finale.

1.5 Conclusion :

Ce chapitre a permis de présenter le contexte général du projet, la problématique liée à la sécurité des réseaux IoMT hospitaliers ainsi que les objectifs visés. Il a mis en évidence l'importance de surveiller localement le trafic réseau des dispositifs médicaux connectés afin

de détecter rapidement les comportements anormaux. La méthodologie adoptée a également été présentée afin de structurer les différentes étapes du travail.

Chapitre 2 : Dataset, sélection du dataset et benchmark

2.1 Choix du dataset :

Le choix du dataset constitue une étape essentielle dans ce projet, car la qualité des données influence directement la pertinence des modèles de détection d'intrusions. Pour ce travail, nous avons choisi le dataset **CICIoMT2024**, développé pour l'étude de la cybersécurité dans les environnements IoMT. Ce choix est cohérent avec notre sujet, qui porte sur le monitoring de la sécurité réseau dans un hôpital connecté.

Contrairement aux datasets génériques de détection d'intrusions, CICIoMT2024 est orienté vers les communications des dispositifs médicaux connectés. Il permet donc de travailler sur des données plus proches d'un environnement hospitalier réel. Le dataset contient du trafic normal ainsi que plusieurs familles d'attaques réseau, notamment des attaques DoS, DDoS, du spoofing, du scan et des activités de reconnaissance.

Cette diversité est importante, car elle permet d'évaluer les modèles dans deux configurations complémentaires : une classification binaire, qui consiste à distinguer le trafic normal du trafic malveillant, et une classification multi-classe, qui cherche à identifier le type précis d'attaque. Le dataset permet ainsi de tester la capacité des modèles à détecter rapidement une anomalie, mais aussi à fournir une information plus détaillée sur la nature de la menace.

Enfin, le volume important de CICIoMT2024 permet de construire un échantillon d'entraînement représentatif tout en respectant les contraintes du projet. L'entraînement est réalisé sur PC à partir d'un échantillon stratifié, puis seuls les modèles sérialisés sont déployés sur Raspberry Pi pour l'inférence. Ce choix permet de concilier richesse des données, pertinence thématique et faisabilité du déploiement Edge

2.2 Description et selection :

Le dataset **CICIoMT2024** est un dataset volumineux contenant plusieurs millions d'enregistrements de trafic réseau IoMT. Il regroupe différentes catégories de trafic, notamment du trafic normal (*benign*) et plusieurs types d'attaques réseau. Cette richesse permet d'entraîner des modèles de détection plus robustes, mais elle représente également une contrainte importante en termes de temps de traitement, de mémoire et de puissance de calcul.

Pour cette raison, l'entraînement des modèles n'a pas été réalisé directement sur le Raspberry Pi. En effet, ce dernier dispose de ressources limitées et n'est pas adapté à l'entraînement d'un

dataset aussi volumineux. L'étape d'apprentissage a donc été effectuée sur PC, tandis que le Raspberry Pi a été utilisé uniquement pour la phase d'inférence et de déploiement Edge.

Afin de garder un volume de données exploitable tout en conservant la représentativité du dataset original, un échantillon stratifié de 300 000 lignes a été créé. L'échantillonnage stratifié permet de préserver la proportion des différentes classes présentes dans le dataset complet. Ainsi, les classes majoritaires et minoritaires restent représentées de manière cohérente, ce qui évite de biaiser l'entraînement des modèles.

Cette sélection permet donc de trouver un compromis entre la qualité des données, le temps d'exécution et la faisabilité technique du projet. Elle garantit également que les résultats obtenus restent exploitables pour comparer les modèles et préparer leur déploiement sur Raspberry Pi.

2.2.1 Visualisation du dataset et distribution des classes :

Afin de mieux comprendre la structure des données, une visualisation du fichier CSV est intégrée dans cette partie. Elle permet d'observer les features utilisées par les modèles, comme les caractéristiques réseau, les indicateurs de protocole, les statistiques de paquets et la colonne label.

Figure 3 : Extrait des premières features du dataset CICIoMT2024

	Header_Length	Protocol Type	Duration	Rate	Srate	Drate	fin_flag_number	syn_flag_number	rst_flag_number	psh_flag_number	...	Std	Tot size	IAT
0	36107.0	17.0	64.0	23221.260	23221.260	0	0.0	0.0	0.0	0.0	...	0.0	50.0	84696600.0
1	12054.0	17.0	64.0	22576.410	22576.410	0	0.0	0.0	0.0	0.0	...	0.0	50.0	84696610.0
2	19762.0	17.0	64.0	19210.564	19210.564	0	0.0	0.0	0.0	0.0	...	0.0	50.0	84696610.0
3	25607.0	17.0	64.0	10879.826	10879.826	0	0.0	0.0	0.0	0.0	...	0.0	50.0	84696610.0
4	12339.5	17.0	64.0	90460.990	90460.990	0	0.0	0.0	0.0	0.0	...	0.0	50.0	84696610.0
5	32581.0	17.0	64.0	14802.994	14802.994	0	0.0	0.0	0.0	0.0	...	0.0	50.0	84696616.0
6	15037.0	17.0	64.0	18609.650	18609.650	0	0.0	0.0	0.0	0.0	...	0.0	50.0	84696610.0
7	24691.0	17.0	64.0	20539.613	20539.613	0	0.0	0.0	0.0	0.0	...	0.0	50.0	84696610.0
8	22115.0	17.0	64.0	32175.740	32175.740	0	0.0	0.0	0.0	0.0	...	0.0	50.0	84696616.0
9	13873.5	17.0	64.0	10191.665	10191.665	0	0.0	0.0	0.0	0.0	...	0.0	50.0	84696610.0

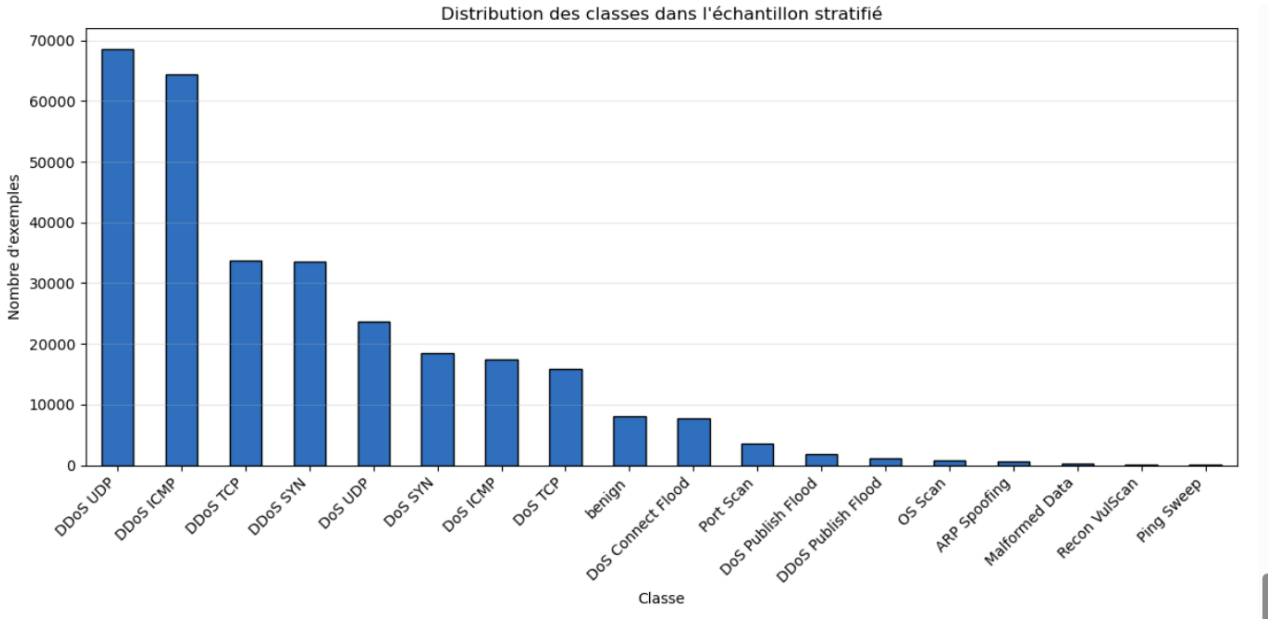
Number	Magnitue	Radius	Covariance	Variance	Weight	label
9.5	10.0	0.0	0.0	0.0	141.5	DDoS UDP
9.5	10.0	0.0	0.0	0.0	141.5	DDoS UDP
9.5	10.0	0.0	0.0	0.0	141.5	DDoS UDP
9.5	10.0	0.0	0.0	0.0	141.5	DDoS UDP
9.5	10.0	0.0	0.0	0.0	141.5	DDoS UDP
9.5	10.0	0.0	0.0	0.0	141.5	DDoS UDP
9.5	10.0	0.0	0.0	0.0	141.5	DDoS UDP
9.5	10.0	0.0	0.0	0.0	141.5	DDoS UDP
9.5	10.0	0.0	0.0	0.0	141.5	DDoS UDP
9.5	10.0	0.0	0.0	0.0	141.5	DDoS UDP

Cette figure présente la structure générale du dataset CICIoMT2024 utilisé dans ce projet. Chaque ligne du dataset représente un flux réseau IoMT, tandis que les colonnes correspondent aux caractéristiques extraites de ce flux. Ces caractéristiques décrivent différents aspects du trafic réseau, comme la longueur des paquets, le type de protocole, la durée de communication, le débit, les flags réseau et les statistiques liées aux paquets échangés.

Le dataset contient également des variables statistiques telles que la moyenne, l'écart-type, la variance, la covariance et le poids du flux. Ces informations permettent de représenter le comportement du trafic de manière numérique afin qu'il puisse être exploité par les modèles de Machine Learning.

La colonne label constitue la variable cible du dataset. Elle indique la classe associée à chaque flux réseau, par exemple un trafic normal ou un type d'attaque. Cette structure permet donc d'utiliser le dataset pour deux types de classification : une classification binaire, qui distingue le trafic normal du trafic malveillant, et une classification multi-classe, qui cherche à identifier le type précis d'attaque.

Figure 4 : Distribution des classes dans l'échantillon stratifié CICIoMT2024



Cette figure présente la distribution des classes dans l'échantillon stratifié de 300 000 lignes extrait du dataset CICIoMT2024. L'axe horizontal représente les différentes classes de trafic réseau, tandis que l'axe vertical indique le nombre d'exemples disponibles pour chaque classe.On observe un déséquilibre important entre les classes. Les attaques DDoS UDP et DDoS ICMP sont les plus représentées, avec environ 60 000 à 70 000 exemples chacune. Elles sont suivies par d'autres attaques fréquentes comme DDoS TCP, DDoS SYN, DoS UDP, DoS SYN, DoS ICMP et DoS TCP.

À l'inverse, certaines classes sont très peu représentées, comme Ping Sweep, Recon VulScan, Malformed Data, ARP Spoofing et OS Scan. Ces classes minoritaires sont plus difficiles à apprendre pour les modèles, car elles contiennent moins d'exemples pendant l'entraînement.

Cette distribution montre que le dataset est naturellement déséquilibré, ce qui est courant dans les jeux de données de cybersécurité. L'échantillonnage stratifié permet de réduire la taille du dataset tout en conservant les proportions initiales des classes. Ainsi, l'échantillon reste représentatif du dataset complet et peut être utilisé pour entraîner et évaluer les modèles dans des conditions réalistes.

2.3 Benchmark :

Afin d'évaluer la solution la plus adaptée au déploiement sur Raspberry Pi, un benchmark a été réalisé sur plusieurs modèles de détection et plusieurs stratégies de transmission. L'objectif de ce benchmark est de comparer les approches non seulement selon leur performance de détection, mais aussi selon leur coût en ressources, notamment le temps d'exécution, la mémoire utilisée et l'énergie estimée.

Pour la partie détection, trois modèles ont été retenus. Le premier est un modèle simple basé sur des seuils (*Thresholding*), utilisé comme méthode de référence. Il permet d'avoir une solution très légère, mais généralement moins précise. Le deuxième modèle est la régression logistique, qui représente une approche linéaire simple, rapide et peu coûteuse. Le troisième modèle est l'arbre de décision, choisi pour sa capacité à capturer des relations non linéaires tout en restant léger et interprétable.

Les modèles sont évalués dans deux configurations. La première est une classification binaire, dont le but est de distinguer le trafic normal du trafic malveillant. La deuxième est une classification multi-classe, qui permet d'identifier plus précisément le type d'attaque détectée. Cette double comparaison permet d'analyser si un modèle est plus adapté à une détection rapide ou à une classification détaillée des menaces.

En parallèle, trois stratégies de transmission sont comparées au niveau de la couche réseau. La stratégie S1 transmet chaque flux immédiatement, la stratégie S2 regroupe les données en lots avant transmission, et la stratégie S3 transmet uniquement les flux détectés comme anomalies. Ces stratégies sont évaluées selon le nombre de messages transmis, la latence, la bande passante et l'énergie logicielle estimée.

Ce benchmark permet donc de justifier le choix final du modèle et de la stratégie de transmission à partir de mesures concrètes, plutôt que sur une simple intuition. L'objectif est d'identifier la solution offrant le meilleur compromis entre précision de détection, rapidité d'exécution et efficacité énergétique sur Raspberry Pi.

2.4 Conclusion :

Ce chapitre a présenté le dataset CICIoMT2024 utilisé pour l'entraînement et l'évaluation des modèles. L'analyse de sa structure et de la distribution des classes a montré que les données sont déséquilibrées, ce qui est fréquent dans les jeux de données de cybersécurité. L'échantillonnage stratifié a permis de réduire la taille du dataset tout en conservant la représentativité des classes, ce qui rend l'entraînement plus efficace et plus réaliste.

Chapitre 3 : Entrainement, Energie, modèles et amélioration :

3.1 Préparation des données :

Avant l'entraînement des modèles, une étape de préparation des données a été réalisée afin de rendre le dataset exploitable par les algorithmes de Machine Learning. Les données issues du dataset CICIoMT2024 contiennent plusieurs caractéristiques numériques décrivant les flux réseau, comme la durée du flux, le protocole utilisé, le débit, le nombre de paquets, ou encore certaines informations liées aux flags réseau.

Dans un premier temps, les valeurs manquantes, infinies ou non exploitables sont traitées afin d'éviter les erreurs pendant l'apprentissage. Les colonnes utilisées comme variables d'entrée sont ensuite converties en format numérique. Cette étape est importante, car les modèles utilisés nécessitent des données structurées et cohérentes pour produire des prédictions fiables.

Les labels ont été préparés selon deux configurations. La première configuration est **binaire**, où les flux sont classés en deux catégories : Normal pour le trafic bénin et Attack pour le trafic malveillant. La deuxième configuration est **multi-classe**, où chaque type d'attaque est conservé comme une classe séparée. Cette double préparation permet de comparer une détection simple des attaques avec une classification plus détaillée des menaces.

3.2 Entrainement des modèles :

L'entraînement des modèles a été réalisé sur PC avec l'environnement Anaconda, afin d'éviter de surcharger le Raspberry Pi. En effet, le dataset utilisé est volumineux et nécessite plus de ressources que celles disponibles sur un dispositif embarqué. Le Raspberry Pi est donc réservé à la phase d'inférence, c'est-à-dire l'utilisation des modèles déjà entraînés pour prédire de nouveaux flux.

Trois modèles ont été entraînés et comparés : un modèle basé sur des seuils (*Thresholding*), une régression logistique et un arbre de décision. Le modèle de seuil sert de référence simple, tandis que la régression logistique et l'arbre de décision permettent d'évaluer des approches d'apprentissage automatique plus performantes tout en restant légères.

Après l'entraînement, les modèles sont évalués à l'aide de plusieurs métriques, notamment l'accuracy, l'AUC, la matrice de confusion et les scores par classe. Les modèles retenus sont ensuite exportés sous forme de fichiers sérialisés avec joblib. Ces fichiers sont copiés sur le Raspberry Pi afin d'être chargés directement pendant la phase d'inférence, sans refaire l'entraînement sur l'équipement embarqué.

3.2.1 Environnement logiciel et système d'exploitation :

L'entraînement des modèles est réalisé sur PC avec Anaconda afin d'éviter de surcharger le Raspberry Pi avec le dataset complet. Les modèles sont ensuite exportés sous forme de fichiers sérialisés puis transférés vers le Raspberry Pi pour l'inférence.

Le Raspberry Pi utilisé pour les tests fonctionne sous Raspbian GNU/Linux 13 (trixie), avec une architecture aarch64. Cette information est importante, car le système d'exploitation et l'architecture influencent la disponibilité des bibliothèques Python, les performances et la consommation des ressources.

Figure 5 : Préparation du dataset pour la classification binaire

```
Shape dataset : (300000, 46)
Features : (300000, 45)
Distribution binaire :
label
Attack    291925
Normal      8075
Name: count, dtype: int64
```

Cette capture présente une étape de préparation du dataset avant l'entraînement des modèles de Machine Learning. L'échantillon utilisé contient 300 000 lignes et 46 colonnes. Parmi ces colonnes, 45 sont utilisées comme features d'entrée, tandis que la colonne restante correspond à la variable cible label.

Dans cette étape, les labels du dataset sont transformés en deux catégories afin de réaliser une classification binaire. La classe Normal correspond au trafic bénin, tandis que la classe Attack regroupe l'ensemble des attaques présentes dans le dataset. Cette transformation permet de simplifier le problème en distinguant uniquement le trafic normal du trafic malveillant.

La distribution affichée montre que l'échantillon contient 291 925 flux d'attaque et 8 075 flux normaux. Ce déséquilibre reflète la nature du dataset CICIoMT2024, où les attaques sont beaucoup plus représentées que le trafic bénin. Malgré ce déséquilibre, cette configuration est utile pour entraîner des modèles capables de détecter rapidement si un flux réseau est normal ou suspect.

Figure 6 : Mesure logicielle du temps, CPU, RAM et énergie

step	test_operation
time_ms	109.4526
cpu_percent	1.0
ram_before_mb	397.417969
ram_after_mb	397.417969
ram_used_mb	0
energy_joules	0.007005

Cette capture présente un exemple de mesure logicielle réalisée pour une étape du pipeline. La mesure repose sur deux outils principaux : time.perf_counter() pour calculer le temps d'exécution et psutil pour relever l'utilisation du CPU ainsi que la mémoire RAM consommée par le processus. La métrique time_ms indique le temps nécessaire pour exécuter l'opération, exprimé en millisecondes. Dans cet exemple, l'étape mesurée prend environ 109.45 ms. La métrique cpu_percent représente le pourcentage d'utilisation du processeur pendant l'exécution, tandis que ram_before_mb et ram_after_mb indiquent la mémoire utilisée avant et après l'étape.

La différence entre ram_after_mb et ram_before_mb permet d'obtenir ram_used_mb, c'est-à-dire la mémoire supplémentaire consommée par l'opération. Dans cet exemple, la valeur est nulle, ce qui signifie que l'étape n'a pas entraîné d'augmentation mesurable de la mémoire. Enfin, energy_joules représente l'énergie logicielle estimée à partir du temps d'exécution, de l'utilisation CPU et de la puissance nominale du Raspberry Pi 4.

Cette méthode de mesure est ensuite appliquée aux différentes étapes du pipeline, comme le chargement du modèle, le preprocessing, l'inférence et l'écriture JSON. Elle permet de comparer les étapes selon leur coût en temps, mémoire et énergie.

Figure 7 : Modèles binaires exportés au format joblib

```
Fichiers modèles binaires :
outputs_iomt\models_binary\decision_tree_binary.joblib
outputs_iomt\models_binary\logistic_regression_binary.joblib
outputs_iomt\models_binary\thresholding_binary.joblib
```

Cette capture montre les fichiers des modèles binaires sauvegardés après la phase d'entraînement.
Lesmodèles decision_tree_binary, logistic_regression_binary et thresholding_binary sont exportés sous forme de fichiers .joblib.

Cette étape est importante, car l'entraînement est réalisé sur PC afin d'éviter de surcharger le Raspberry Pi avec un dataset volumineux. Les fichiers sérialisés sont ensuite transférés vers le Raspberry Pi, où ils sont uniquement utilisés pour la phase d'inférence. Cette approche permet de respecter les contraintes d'un environnement Edge en limitant la charge de calcul sur le dispositif embarqué.

Figure 8 : Export des modèles multi-classes au format joblib

```
Fichiers modèles multi-classes :
outputs_iomt\models_multiclass\decision_tree_multiclass.joblib
outputs_iomt\models_multiclass\logistic_regression_multiclass.joblib
```

Cette figure illustre l'étape d'exportation des modèles multi-classes après leur entraînement sur PC. Le but de cette étape est de sauvegarder le modèle entraîné dans un fichier sérialisé afin de pouvoir le réutiliser plus tard sans refaire l'entraînement.

Dans ce projet, l'entraînement est effectué sur PC avec Anaconda, car le dataset CICIoMT2024 est volumineux et nécessite plus de ressources que celles disponibles sur Raspberry Pi. Une fois le modèle entraîné et évalué, il est exporté au format .joblib. Ce format permet de conserver le modèle, ses paramètres appris, ainsi que les informations nécessaires à son utilisation, comme la liste des features et les classes.

Les modèles multi-classes permettent d'aller au-delà de la simple détection Normal ou Attack. Ils cherchent à identifier le type précis d'attaque réseau, par exemple DDoS UDP, DDoS ICMP, DoS TCP, Port Scan ou encore ARP Spoofing. Cette classification plus détaillée est utile

pour l'analyse de sécurité, car elle permet de mieux comprendre la nature de la menace détectée.

Après l'export, les fichiers .joblib sont transférés vers le Raspberry Pi. Lors du déploiement, le Raspberry Pi charge uniquement ces fichiers et réalise l'inférence sur les nouveaux flux réseau. Cette approche respecte les contraintes Edge, car elle évite d'exécuter l'entraînement sur le Raspberry Pi et limite son rôle au chargement du modèle et à la prédiction.

3.3 Résultats des modèles :

Tableau 1 : Comparaison des performances et de la taille des modèles de classification

short_name	Model	task	accuracy	auc	model_size_kb
TH-Bin	thresholding	binary	0.4589	0.6594	0.76
LR-Bin	logistic_regression	binary	0.9886	0.9974	4.94
DT-Bin	decision_tree	binary	0.994	0.9977	9.76
LR-Multi	logistic_regression	multiclass	0.5822	0.9674	11.59
DT-Multi	decision_tree	multiclass	0.9708	0.9984	21.41

Ce tableau présente les performances des différents modèles entraînés et comparés dans le projet. La colonne short_name donne un nom abrégé pour chaque expérience, tandis que la colonne model indique le type de modèle utilisé. La colonne task précise si le modèle est évalué en classification binaire ou en classification multi-classe.

Les résultats montrent que le modèle basé sur des seuils (TH-Bin) obtient les performances les plus faibles, avec une accuracy de 0.4589 et une AUC de 0.6594. Ce résultat confirme que les règles simples ne sont pas suffisantes pour détecter correctement les anomalies dans un trafic réseau IoMT complexe.

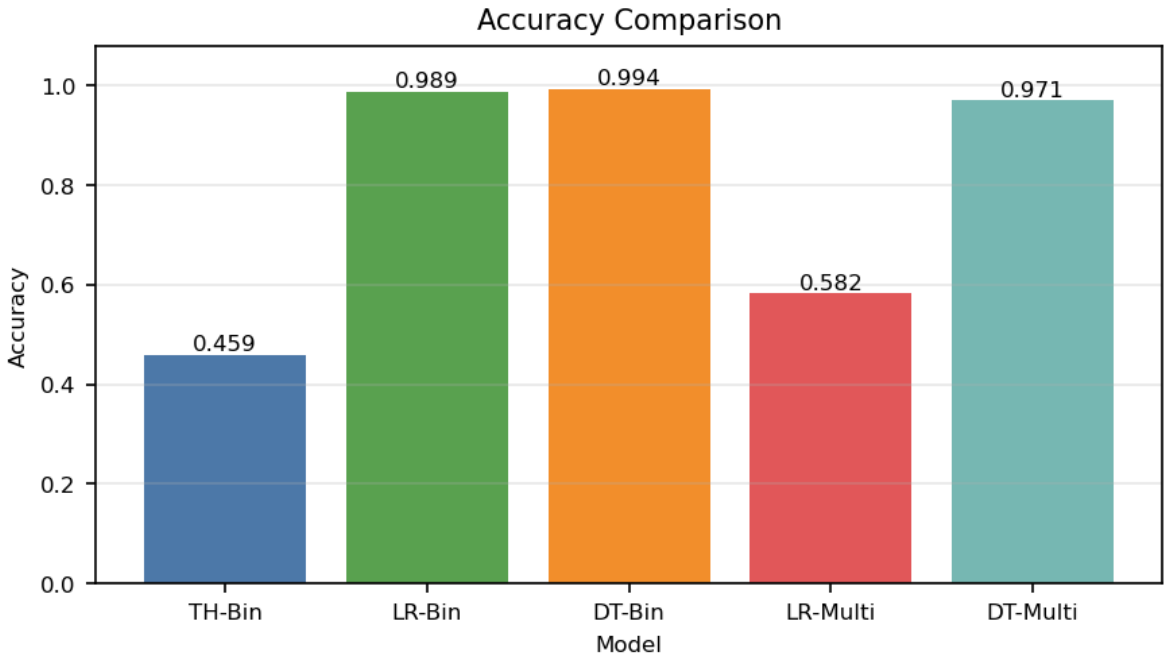
En classification binaire, la régression logistique (LR-Bin) et l'arbre de décision (DT-Bin) obtiennent de très bons résultats. L'arbre de décision atteint la meilleure accuracy avec 0.9940 et une AUC de 0.9977, ce qui montre sa capacité à distinguer efficacement le trafic normal du trafic malveillant.

En classification multi-classe, la régression logistique (LR-Multi) présente une accuracy plus faible, ce qui indique qu'elle confond plusieurs types d'attaques. En revanche, l'arbre de

décision multi-classe (DT-Multi) atteint une accuracy de 0.9708 et une AUC de 0.9984, ce qui en fait le modèle le plus performant pour identifier les différentes catégories d'attaques.

Enfin, la colonne model_size_kb montre que tous les modèles restent très légers, avec des tailles inférieures à 22 Ko. Cela confirme qu'ils sont adaptés à un déploiement sur Raspberry Pi, où les ressources de stockage et de mémoire sont limitées. Globalement, le modèle Decision Tree offre le meilleur compromis entre précision, capacité de classification et légèreté.

Figure 9 : Comparaison des performances des modèles selon l'accuracy



Cette figure compare les performances des différents modèles selon la métrique accuracy. Les modèles évalués sont le Thresholding binaire (TH-Bin), la Logistic Regression binaire (LR-Bin), le Decision Tree binaire (DT-Bin), la Logistic Regression multi-classe (LR-Multi) et le Decision Tree multi-classe (DT-Multi).

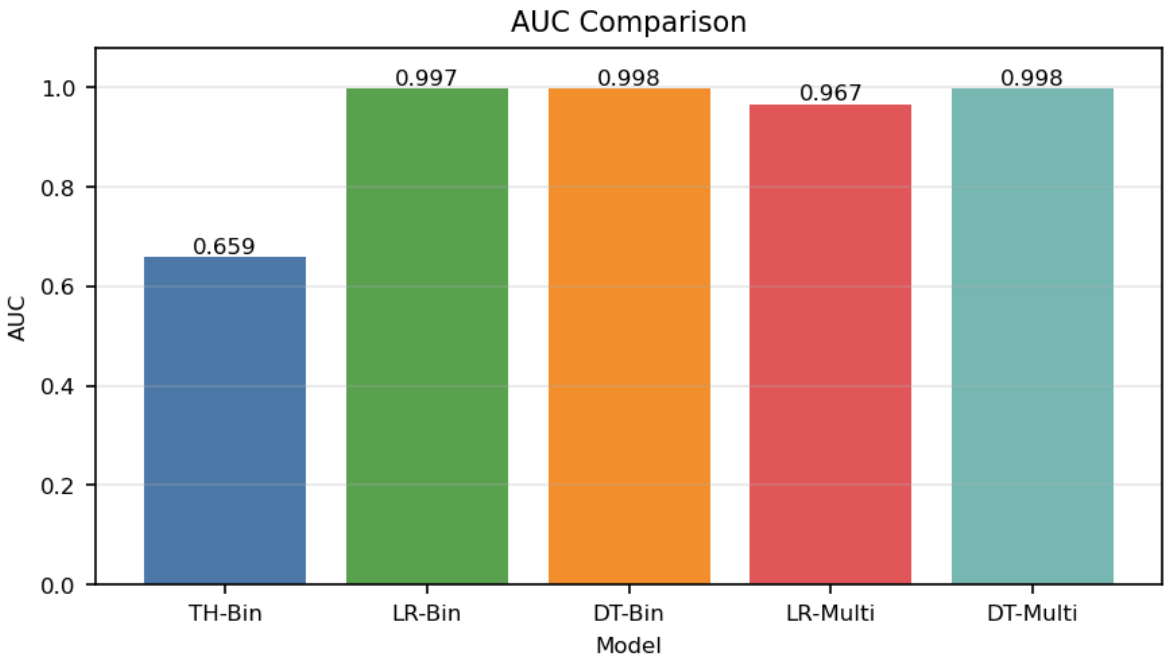
On remarque que le modèle TH-Bin obtient le score le plus faible, avec une accuracy de 0.459. Cela montre qu'une approche basée uniquement sur des seuils simples n'est pas suffisante pour détecter efficacement les anomalies réseau dans ce dataset. Ce type de modèle reste léger, mais il ne capture pas correctement la complexité des comportements présents dans le trafic IoMT. Les modèles binaires de Machine Learning donnent de bien meilleurs résultats. La Logistic Regression binaire atteint une accuracy de 0.989, tandis que le Decision Tree binaire

obtient la meilleure performance avec une accuracy de 0.994. Cela montre que ces modèles distinguent très efficacement le trafic normal du trafic d'attaque.

Pour la classification multi-classe, la Logistic Regression obtient une accuracy plus faible, égale à 0.582. Cette baisse s'explique par la difficulté supplémentaire du problème multi-classe, où le modèle doit identifier le type précis d'attaque et non seulement distinguer Normal et Attack. En revanche, le Decision Tree multi-classe atteint une accuracy de 0.971, ce qui montre qu'il reste performant même dans une configuration plus complexe.

Globalement, ce graphe montre que le Decision Tree offre le meilleur compromis en termes de performance. Le DT-Bin est le plus adapté pour une détection rapide Normal/Attack, tandis que le DT-Multi permet une analyse plus détaillée du type d'attaque avec une accuracy élevée.

Figure 10 : Comparaison de l AUC des modèles



Cette figure compare les modèles selon la métrique AUC associée à la courbe ROC. L'AUC mesure la capacité d'un modèle à séparer correctement les classes. Plus la valeur est proche de 1, plus le modèle distingue efficacement les flux normaux des flux malveillants, ou les différentes classes dans le cas multi-classe.

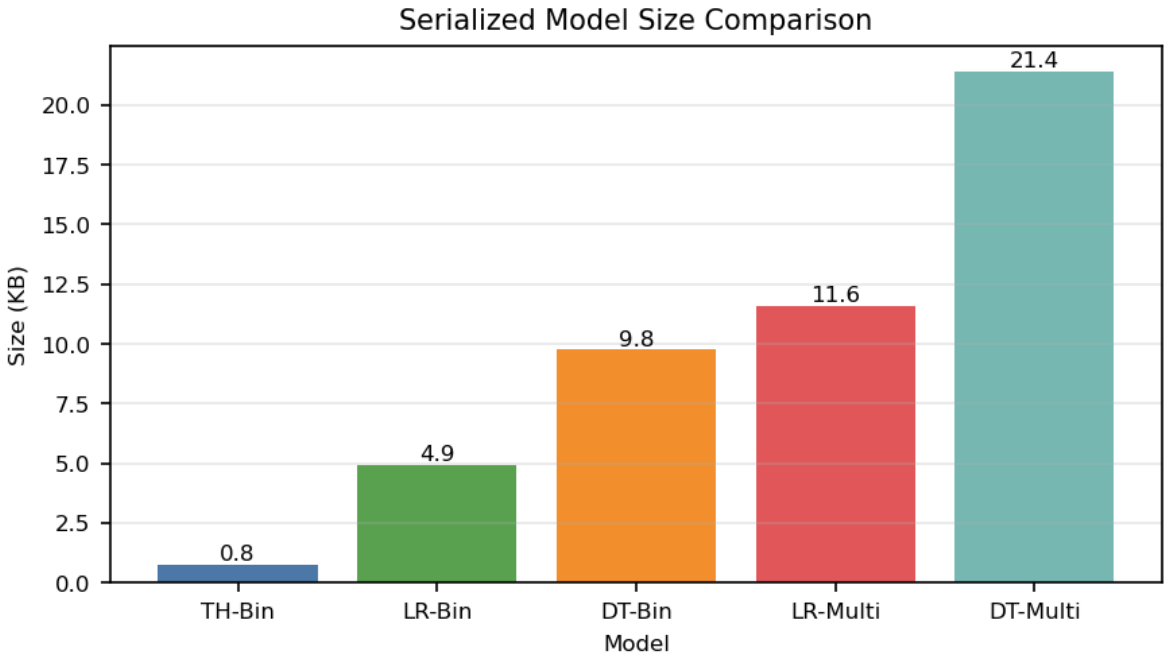
Le modèle TH-Bin obtient l'AUC la plus faible, avec une valeur de 0.659. Ce résultat confirme que l'approche par seuils simples n'est pas suffisamment robuste pour ce dataset. Elle permet une première détection basique, mais elle ne sépare pas correctement les comportements normaux et les attaques. Les modèles binaires de Machine Learning obtiennent des résultats très élevés. La Logistic Regression binaire atteint une AUC de 0.997, tandis que le Decision Tree binaire atteint environ 0.998. Ces valeurs

montrent que les deux modèles sont capables de distinguer très efficacement le trafic Normal du trafic Attack.

En multi-classe, la Logistic Regression obtient une AUC de 0.967, ce qui reste un bon résultat malgré une accuracy plus faible. Cela signifie que le modèle conserve une bonne capacité de séparation globale, même s'il commet davantage d'erreurs sur certaines classes. Le Decision Tree multi-classe atteint également une AUC d'environ 0.998, ce qui montre une très bonne capacité à différencier les classes d'attaques.

Globalement, ce graphe confirme que les modèles basés sur Machine Learning, en particulier les Decision Trees, sont beaucoup plus performants que le seuil simple. Le Decision Tree binaire et le Decision Tree multi-classe présentent les meilleurs scores AUC, ce qui justifie leur choix pour la suite du projet.

Figure 11 : Comparaison de la taille des modèles sérialisés



Cette figure compare la taille des fichiers modèles après sérialisation. La taille du modèle est un critère important dans un environnement Edge, car un modèle plus léger consomme généralement moins d'espace de stockage et peut être chargé plus rapidement en mémoire.

Le modèle TH-Bin est le plus léger, avec une taille d'environ 0.8 KB. Cela s'explique par sa simplicité, car il repose uniquement sur une logique de seuils. Cependant, ses performances

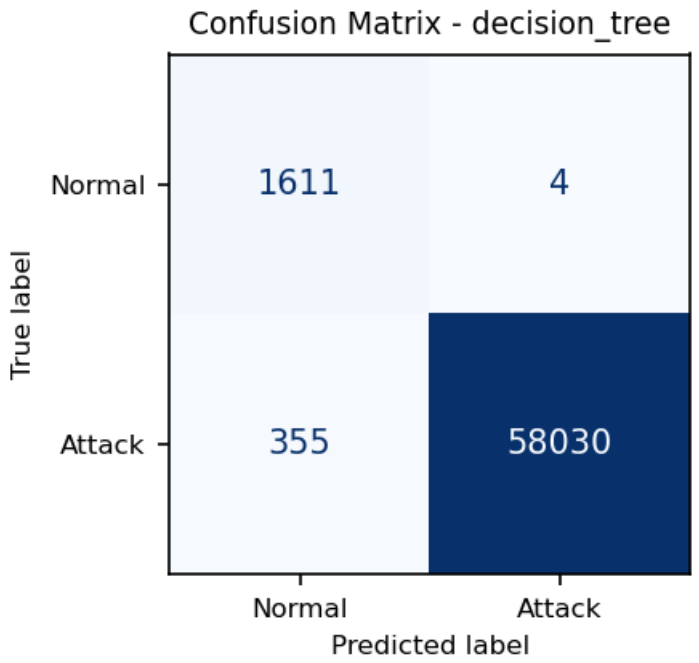
en accuracy et en AUC restent faibles par rapport aux autres modèles, ce qui limite son intérêt pour une détection fiable.

La Logistic Regression binaire (LR-Bin) reste également très légère, avec une taille d'environ 4.9 KB. Le Decision Tree binaire (DT-Bin) est légèrement plus volumineux, avec environ 9.8 KB, mais il offre les meilleures performances en classification binaire. Cette taille reste très faible et adaptée à un déploiement sur Raspberry Pi.

Les modèles multi-classes sont plus volumineux, car ils doivent apprendre à distinguer plusieurs types d'attaques au lieu de séparer uniquement Normal et Attack. La Logistic Regression multi-classe atteint environ 11.6 KB, tandis que le Decision Tree multi-classe est le plus grand avec environ 21.4 KB. Malgré cela, sa taille reste raisonnable pour un dispositif Edge.

Globalement, ce graphe montre que tous les modèles sérialisés restent légers. Le Decision Tree binaire représente le meilleur compromis entre taille réduite et performance élevée, tandis que le Decision Tree multi-classe reste acceptable lorsqu'une identification plus détaillée du type d'attaque est nécessaire.

Figure 12 : Matrice de confusion Decision Tree binaire



Cette figure présente la matrice de confusion du modèle Decision Tree en classification binaire. L'objectif de ce modèle est de distinguer deux classes : Normal, qui correspond au trafic bénin, et Attack, qui correspond au trafic malveillant.

Les lignes de la matrice représentent les vraies classes, tandis que les colonnes représentent les classes prédites par le modèle. Ainsi, la diagonale principale indique les prédictions correctes, alors que les autres cases correspondent aux erreurs de classification.

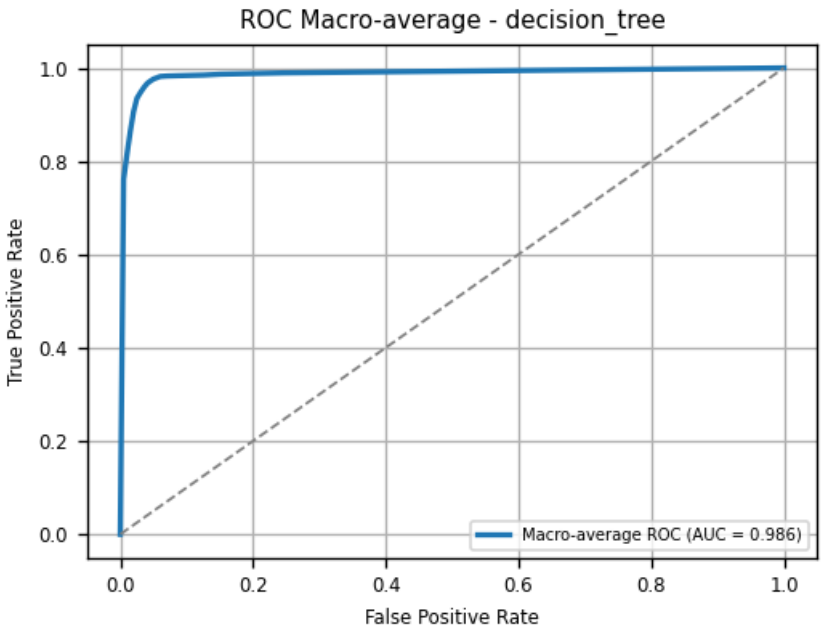
Le modèle a correctement classé 1611 flux normaux comme Normal et 58030 flux d'attaque comme Attack. Ces deux valeurs se trouvent sur la diagonale de la matrice, ce qui montre que la majorité des prédictions sont correctes.

La matrice montre également 4 flux normaux classés à tort comme Attack. Ces erreurs correspondent à des faux positifs. Dans un système de sécurité, cela signifie qu'un trafic normal peut être signalé comme suspect. Ce type d'erreur peut générer une alerte inutile, mais il reste limité dans ce résultat.

On observe aussi 355 flux d'attaque classés à tort comme Normal. Ces erreurs correspondent à des faux négatifs. Elles sont plus critiques dans un contexte de cybersécurité, car elles signifient que certaines attaques ne sont pas détectées par le système. Cependant, ce nombre reste faible par rapport aux 58030 attaques correctement détectées.

Globalement, cette matrice confirme les bonnes performances du modèle Decision Tree binaire. Le modèle détecte correctement la grande majorité des attaques tout en conservant un très faible nombre d'erreurs sur le trafic normal. Ces résultats justifient son choix comme modèle principal pour la détection rapide Normal/Attack.

Figure 13 : Courbe ROC macro-moyenne du modèle Decision Tree multi-classe



Cette figure présente la courbe ROC macro-moyenne du modèle Decision Tree en classification multi-classe. La courbe ROC permet d'évaluer la capacité du modèle à distinguer correctement

les classes. L'axe horizontal représente le taux de faux positifs, tandis que l'axe vertical représente le taux de vrais positifs.

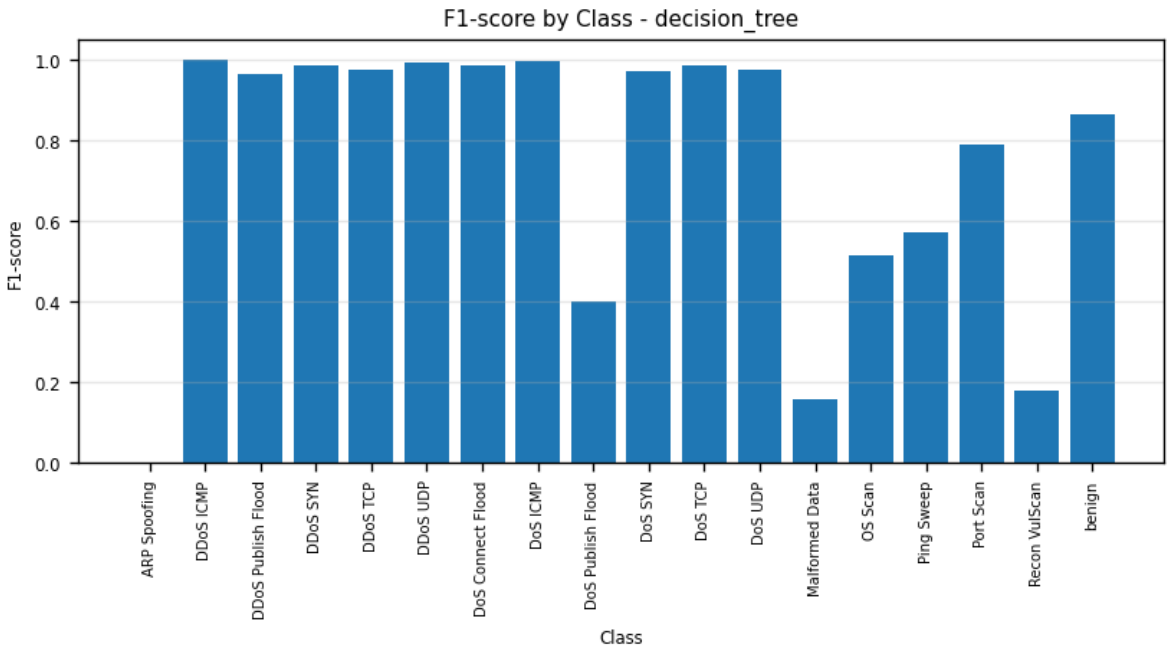
Dans le cas multi-classe, la courbe ROC est calculée pour chaque classe, puis une moyenne macro est réalisée. Cette moyenne donne une vision globale des performances du modèle sur l'ensemble des classes, sans se limiter uniquement aux classes les plus représentées.

La courbe bleue est très proche du coin supérieur gauche du graphique. Cela signifie que le modèle obtient un taux élevé de bonnes détections tout en conservant un faible taux de fausses alertes. Plus la courbe est proche de ce coin, meilleure est la capacité de séparation du modèle.

La valeur AUC obtenue est de **0.986**, ce qui indique une très bonne performance globale. Une AUC proche de 1 signifie que le modèle distingue efficacement les différentes classes de trafic réseau, même dans une configuration plus complexe que la classification binaire.

La ligne diagonale grise représente un classifieur aléatoire. Comme la courbe du modèle est largement au-dessus de cette diagonale, cela montre que le Decision Tree multi-classe fournit des prédictions nettement meilleures qu'un choix aléatoire. Ces résultats confirment que ce modèle est pertinent pour identifier plus finement les types d'attaques réseau IoMT.

Figure 14 : F1-score par classe du modèle Decision Tree multi-classe



Cette figure présente le F1-score obtenu par le modèle Decision Tree pour chaque classe dans la configuration multi-classe. Le F1-score combine la précision et le rappel dans une seule métrique. Il est particulièrement utile lorsque le dataset est déséquilibré, car il permet d'évaluer la qualité de détection de chaque classe séparément.

On remarque que plusieurs classes obtiennent des F1-scores très élevés, proches de 1. C'est le cas de classes comme DDoS ICMP, DDoS SYN, DDoS TCP, DDoS UDP, DoS ICMP, DoS SYN, DoS TCP et DoS UDP. Ces résultats montrent que le modèle reconnaît très bien les attaques les plus représentées dans le dataset.

La classe benign obtient également un score relativement élevé, ce qui indique que le modèle arrive à identifier correctement une grande partie du trafic normal. Cela est important dans un système de détection d'intrusions, car il faut à la fois détecter les attaques et éviter de classer trop de trafic normal comme malveillant.

En revanche, certaines classes obtiennent des scores plus faibles, notamment ARP Spoofing, Malformed Data et Recon VulScan. Cette baisse de performance s'explique principalement par le faible nombre d'exemples disponibles pour ces classes dans l'échantillon. Les classes minoritaires sont plus difficiles à apprendre pour le modèle, car elles apparaissent rarement pendant l'entraînement.

Globalement, cette figure montre que le modèle Decision Tree multi-classe est très performant sur les classes principales, mais qu'il rencontre davantage de difficultés sur les classes rares. Ce résultat confirme l'importance d'analyser les performances par classe, et pas seulement l'accuracy globale.

3.4 Mesure énergétique logicielle :

Pour estimer la consommation énergétique de chaque étape du pipeline, une mesure logicielle a été mise en place. Cette mesure repose sur le temps d'exécution, l'utilisation du CPU et une puissance nominale du Raspberry Pi 4 fixée à **6.4 W** sous charge.

La formule utilisée est :

Énergie (J) = temps exécution (s) × 6.4 W × CPU_percent / 100

Le temps d'exécution est mesuré avec time.perf_counter(), qui permet d'obtenir une mesure précise de la durée de chaque opération. L'utilisation du CPU et de la RAM est mesurée avec la bibliothèque psutil. Cette approche permet d'estimer le coût énergétique des différentes étapes, comme la collecte des données, la transmission, le chargement du modèle, l'inférence et l'écriture des résultats JSON.

Même si cette méthode reste une estimation logicielle, elle permet de comparer les étapes entre elles et d'identifier les parties les plus coûteuses du système. Elle est donc utile pour analyser l'efficacité énergétique du pipeline déployé sur Raspberry Pi.

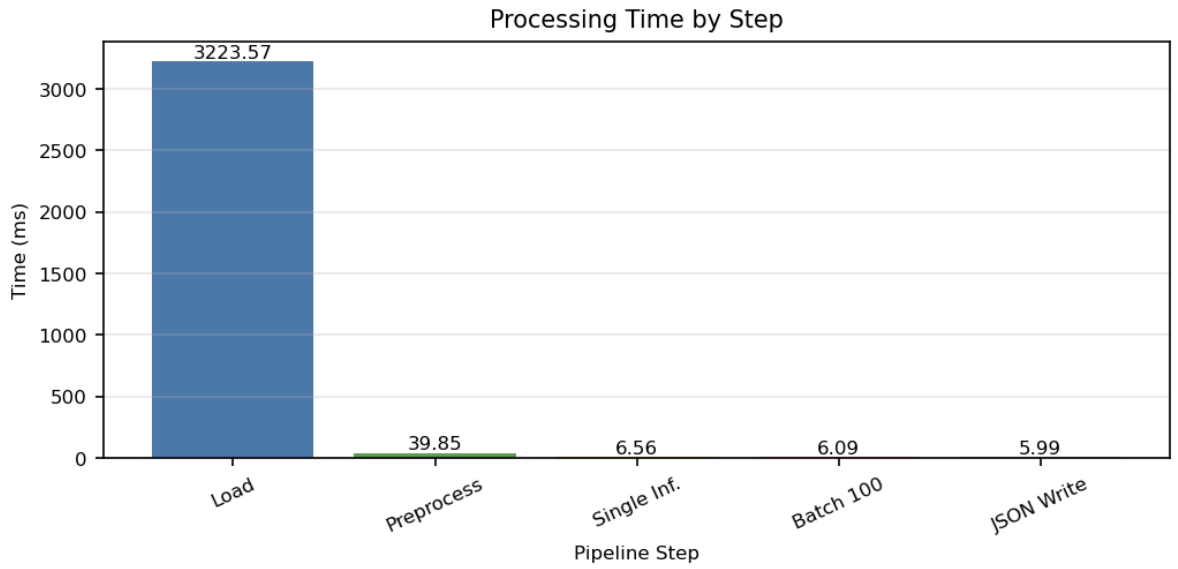
3.5 Améliorations :

À partir des résultats obtenus, le modèle **Decision Tree** a été retenu comme modèle principal du système. Ce choix est justifié par son bon compromis entre précision, légèreté et rapidité d'exécution. En effet, il obtient de très bonnes performances en classification binaire et multi-classe, tout en conservant une taille de fichier faible, ce qui facilite son déploiement sur Raspberry Pi.

Une autre amélioration concerne l'utilisation de l'**inférence en batch**. Au lieu de traiter les flux un par un, plusieurs échantillons sont traités ensemble. Cette approche permet de réduire le coût moyen par prédiction et d'optimiser l'utilisation du CPU. Elle est particulièrement intéressante dans un système IoT où les ressources matérielles sont limitées.

Enfin, le modèle est chargé une seule fois au démarrage du pipeline, puis réutilisé pour les prédictions suivantes. Cette amélioration permet d'éviter de répéter l'étape de chargement, qui est la plus coûteuse en temps et en énergie. Ainsi, le système devient plus efficace pour un fonctionnement continu sur Raspberry Pi.

Figure 15 : Analyse du temps d'exécution des différentes étapes du pipeline de traitement



Cette figure présente le temps d'exécution des principales étapes du pipeline de traitement. Les étapes mesurées sont le chargement du modèle (Load), le preprocessing des données, l'inférence unitaire, l'inférence en batch de 100 échantillons et l'écriture des prédictions au format JSON. On observe que l'étape Load est largement la plus coûteuse, avec un temps d'environ **3223.57 ms**. Cette durée correspond au chargement du modèle sérialisé depuis le

disque et à son initialisation en mémoire. Ce résultat est logique, car le modèle et les objets associés doivent être lus puis reconstruits avant de pouvoir être utilisés pour l'inférence.

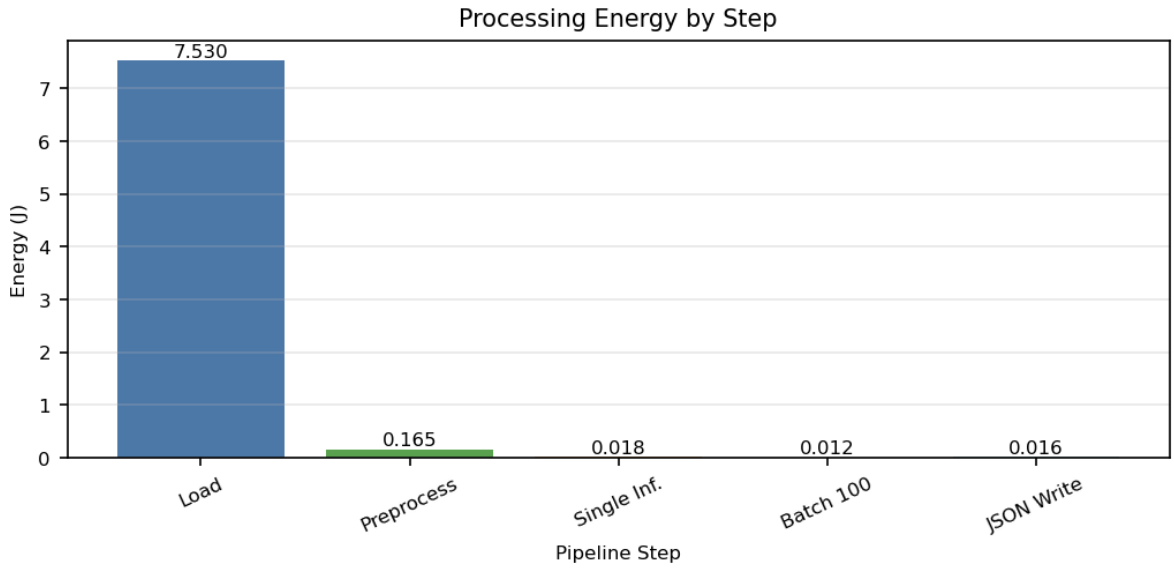
Les autres étapes sont beaucoup plus rapides. Le preprocessing de 100 échantillons prend environ **39.85 ms**, ce qui reste faible par rapport au chargement du modèle. Cette étape consiste à préparer les données avant la prédiction, notamment en sélectionnant les features nécessaires, en appliquant les transformations prévues et en assurant la compatibilité avec le modèle.

L'inférence unitaire prend environ **6.56 ms**, tandis que l'inférence en batch de 100 échantillons prend environ **6.09 ms**. Cela montre que le traitement par batch est plus efficace, car il permet de prédire plusieurs flux en un seul appel au modèle, réduisant ainsi le coût moyen par prédiction.

L'écriture JSON prend environ **5.99 ms**, ce qui montre que la génération du fichier de sortie standardisé reste légère. Cette étape est importante car elle permet de produire des prédictions exploitables par les autres couches du système et par le dashboard.

Globalement, cette figure montre que le coût principal du pipeline vient du chargement initial du modèle. Une fois le modèle chargé, les étapes de preprocessing, d'inférence et d'écriture sont rapides. Il est donc préférable de charger le modèle une seule fois au démarrage du système, puis de le réutiliser pour traiter les flux suivants.

Figure 16 : Énergie consommée par étape du pipeline de traitement



Cette figure présente l'énergie logicielle estimée pour chaque étape du pipeline de traitement. L'énergie est calculée à partir du temps d'exécution, du pourcentage d'utilisation CPU et de la

puissance nominale du Raspberry Pi 4, fixée à **6.4 W**. Cette mesure permet d'identifier les étapes les plus coûteuses du système du point de vue énergétique.

On observe que l'étape Load, correspondant au chargement du modèle, consomme la plus grande partie de l'énergie, avec environ **7.530 J**. Cette valeur est nettement supérieure à celle des autres étapes. Cela s'explique par le fait que le chargement nécessite la lecture du fichier modèle depuis le stockage, la désérialisation du modèle et son initialisation en mémoire.

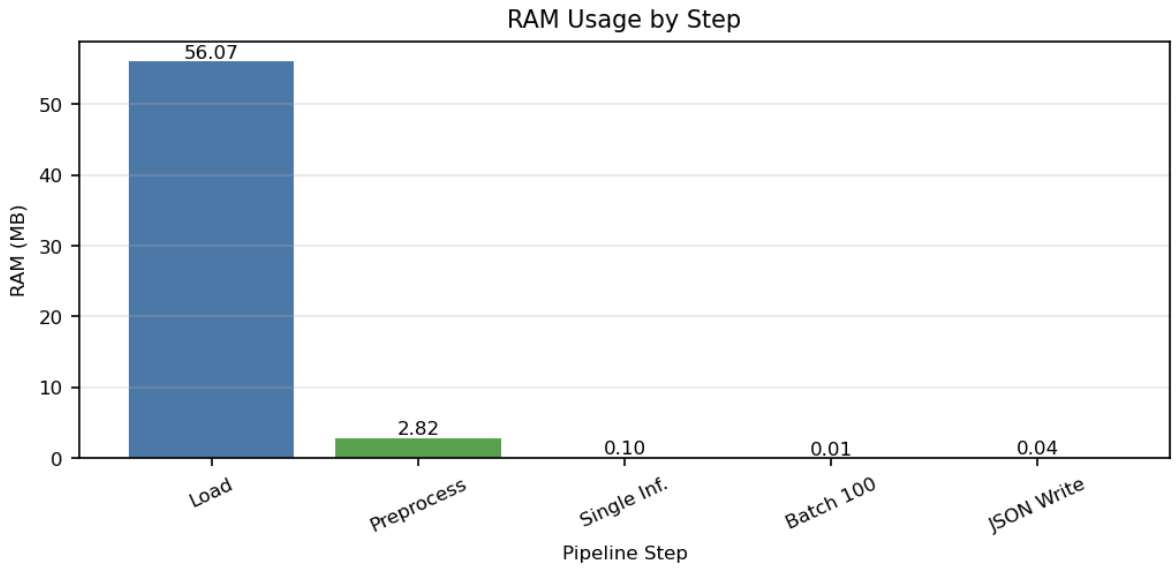
Le preprocessing consomme environ **0.165 J**, ce qui reste faible par rapport au chargement. Cette étape prépare les données avant l'inférence, notamment en appliquant les transformations nécessaires pour que les échantillons soient compatibles avec le modèle entraîné.

Les étapes d'inférence sont beaucoup moins coûteuses. L'inférence unitaire consomme environ **0.018 J**, tandis que l'inférence en batch de 100 échantillons consomme environ **0.012 J**. Ce résultat montre que le traitement en batch est plus efficace énergétiquement, car il permet de traiter plusieurs prédictions avec un coût global très faible.

L'écriture JSON consomme environ **0.016 J**, ce qui montre que la sauvegarde des prédictions dans un format standardisé n'a pas un impact énergétique important sur le pipeline.

Globalement, cette figure confirme que le chargement du modèle est l'étape dominante en énergie. Une bonne stratégie consiste donc à charger le modèle une seule fois au démarrage du système, puis à le réutiliser pour toutes les prédictions suivantes. Cette approche permet de réduire le coût énergétique moyen en fonctionnement continu.

Figure 17 :RAM par étape du pipeline



Cette figure présente la consommation mémoire RAM mesurée pour les différentes étapes du pipeline de traitement. La mémoire est exprimée en mégaoctets (MB) et permet d'évaluer l'impact de chaque étape sur les ressources du système.

On observe que l'étape Load, correspondant au chargement du modèle, est celle qui consomme le plus de mémoire, avec environ **56.07 MB**. Cette consommation est principalement due à la lecture du fichier modèle sérialisé, à sa désérialisation et à son chargement en mémoire pour permettre l'inférence.

Le preprocessing consomme environ **2.82 MB**, ce qui reste relativement faible. Cette mémoire supplémentaire est liée à la préparation des échantillons avant la prédiction, comme la sélection des colonnes, le traitement des valeurs et la création des structures de données nécessaires au modèle.

Les étapes d'inférence consomment très peu de mémoire. L'inférence unitaire utilise environ **0.10 MB**, tandis que l'inférence en batch de 100 échantillons utilise environ **0.01 MB**. Cela montre que, une fois le modèle chargé, la prédiction elle-même est légère en mémoire.

L'écriture JSON consomme environ **0.04 MB**, ce qui indique que la génération du fichier de sortie standardisé n'ajoute presque pas de charge mémoire au système.

Globalement, cette figure confirme que le chargement du modèle représente l'étape la plus coûteuse en RAM. Cependant, après ce chargement initial, les autres étapes du pipeline restent très légères. Cela confirme que le modèle peut être utilisé efficacement dans un environnement Edge, à condition de le charger une seule fois puis de le réutiliser pour les prédictions suivantes.

3.6 Conclusion :

Ce chapitre a détaillé les étapes de préparation des données, d'entraînement et d'évaluation des modèles de Machine Learning. Les résultats ont montré que les modèles Decision Tree offrent les meilleures performances, aussi bien en classification binaire qu'en classification multi-classe. Les mesures de temps, RAM et énergie ont également permis d'identifier les étapes les plus coûteuses du pipeline, notamment le chargement du modèle.

Chapitre 4 : Inférence, Raspberry Pi, résultats des mesures et Dashboard

4.1 Compression locale avant déploiement :

Avant le déploiement sur Raspberry Pi, les modèles entraînés sur PC sont optimisés localement afin de réduire leur taille et de mieux les adapter à un environnement Edge. Cette étape s'inscrit dans une logique TinyML, dont l'objectif est de rendre les modèles plus légers avant leur exécution sur un dispositif embarqué.

Dans ce projet, les modèles sont d'abord exportés au format joblib, puis convertis au format ONNX lorsque le modèle est compatible. ONNX est un format portable adapté à l'inférence et cohérent avec les modèles scikit-learn utilisés, notamment la Logistic Regression et le Decision Tree. Cette conversion est réalisée sur PC avant le transfert vers le Raspberry Pi. Le Raspberry Pi n'est donc pas utilisé pour l'entraînement ni pour la conversion des modèles. Il reçoit uniquement des modèles déjà entraînés et optimisés, puis les utilise pour la phase d'inférence. Cette organisation permet de réduire la charge de calcul sur le dispositif embarqué et de respecter les contraintes de ressources du projet.

Figure 18 : Comparaison de la taille des modèles joblib et ONNX

	modele	taille_joblib_kb	taille_onnx_kb	gain_%
0	logistic_regression_binary	4.94	1.84	62.80
1	decision_tree_binary	9.76	3.93	59.76
2	logistic_regression_multiclass	11.59	5.56	52.06
3	decision_tree_multiclass	21.41	12.04	43.78

Ce tableau présente la comparaison entre la taille des modèles sérialisés au format joblib et leur version convertie au format ONNX. Cette conversion est réalisée localement avant le

déploiement sur Raspberry Pi afin d'obtenir des modèles plus légers et mieux adaptés à un environnement Edge.

On remarque que la conversion ONNX permet de réduire significativement la taille des fichiers. Le modèle logistic_regression_binary passe de **4.94 KB** à **1.84 KB**, soit un gain de **62.80 %**. Le modèle decision_tree_binary passe de **9.76 KB** à **3.93 KB**, avec un gain de **59.76 %**. Ces résultats sont importants, car les modèles binaires sont ceux utilisés pour la détection rapide Normal/Attack.

Pour les modèles multi-classes, la réduction est également notable. Le modèle logistic_regression_multiclass passe de **11.59 KB** à **5.56 KB**, soit un gain de **52.06 %**. Le modèle decision_tree_multiclass passe de **21.41 KB** à **12.04 KB**, avec un gain de **43.78 %**. Même si le gain est légèrement plus faible pour le Decision Tree multi-classe, la taille finale reste adaptée à un déploiement sur Raspberry Pi.

Cette optimisation montre que le format ONNX permet de réduire l'espace de stockage nécessaire et facilite le transfert des modèles vers un dispositif Edge. Elle s'inscrit dans une logique TinyML, où l'objectif est de rendre les modèles plus légers avant leur utilisation sur un système embarqué. Le modèle Thresholding n'a pas été converti, car il repose sur une classe personnalisée, contrairement aux modèles scikit-learn standards compatibles avec ONNX.

4.2 Déploiement Raspberry Pi :

Le projet est déployé sur le Raspberry Pi dans le dossier ~/iot_project. Ce dossier contient les scripts Python nécessaires à l'exécution des différentes couches du système, ainsi que les fichiers de résultats, les modèles sérialisés et les données utilisées pour l'inférence.

Les scripts principaux sont organisés selon les couches de l'architecture IoT. Le fichier pi_sensing_replay.py est utilisé pour rejouer les flux réseau IoMT et simuler la collecte des données. Le script pi_network_layer.py permet de tester les différentes stratégies de transmission. Le fichier pi_processing_inference.py charge le modèle entraîné et effectue les prédictions localement. Enfin, app_dashboard.py lance le dashboard Flask permettant de visualiser les résultats depuis un navigateur.

Ce déploiement permet de reproduire un fonctionnement Edge, où la collecte simulée, la transmission, l'inférence et la visualisation sont exécutées directement sur le Raspberry Pi. Ainsi, le système peut être testé dans des conditions proches d'un environnement IoT embarqué, avec des contraintes de temps, de mémoire et de consommation énergétique.

Figure 19 : Installation et vérification des bibliothèques sur Raspberry Pi

```
saad@IP:~ $ sudo apt install -y python3-flask
python3-flask is already the newest version (3.1.1-1).
python3-flask set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2
saad@IP:~ $ source ~/iot_env/bin/activate
(iot_env) saad@IP:~ $ python -c "import pandas, numpy, sklearn, joblib, psutil, flask; print('OK')"
OK
```

Figure 20 : Structure des fichiers du projet sur Raspberry Pi

```
(iot_env) saad@IP:~/iot_project $ cd ~/iot_project
ls raspberry_scripts
cp raspberry_scripts/*.py .
ls
app_dashboard.py     pi_processing_inference.py
pi_network_layer.py  pi_sensing_replay.py
app_dashboard.py  pi_network_layer.py        pi_sensing_replay.py
outputs_iomt      pi_processing_inference.py  raspberry_scripts
```

4.3 Sensing Layer sur Raspberry Pi :

Figure 21 : Exécution de la couche Sensing Layer sur Raspberry Pi

```
(iot_env) saad@IP:~/iot_project $ python pi_sensing_replay.py
Raspberry Pi - Sensing Layer replay
Input: outputs_iomt/sensing_layer/sensed_flows_balanced_1000.csv
Saved: outputs_iomt/pi_run/sensing_layer/pi_sensed_flows_1000.csv
Metrics: outputs_iomt/pi_run/sensing_layer/pi_sensing_metrics.json
{
    "layer": "sensing",
    "method": "raspberry_dataset_replay",
    "source_file": "outputs_iomt/sensing_layer/sensed_flows_balanced_1000.csv",
    "output_file": "outputs_iomt/pi_run/sensing_layer/pi_sensed_flows_1000.csv",
    "rows_collected": 1000,
    "benign_rows": 500,
    "attack_rows": 500,
    "timestamp": "2026-04-15T14:10:30.642821",
    "time_ms": 129.01158000022406,
    "cpu_percent": 25.5,
    "ram_before_mb": 53.48828125,
    "ram_after_mb": 56.42578125,
    "ram_used_mb": 2.9375,
    "energy_joules_software": 0.21054689856036568,
    "energy_formula": "duration_seconds * 6.4W * cpu_percent/100"
}
```

Cette capture montre l'exécution du script pi_sensing_replay.py sur le Raspberry Pi. Ce script correspond à la couche Sensing Layer du système. Dans ce projet, la collecte réelle par capteurs ou trafic réseau direct est remplacée par une simulation contrôlée à partir d'un fichier extrait du dataset CICIoMT2024.

Le script lit le fichier sensed_flows_balanced_1000.csv, qui contient un ensemble équilibré de 1000 flux réseau. Cet ensemble est composé de 500 flux bénins et de 500 flux d'attaque. Ce choix permet de tester le pipeline dans des conditions équilibrées, afin d'observer le comportement du système face aux deux types de trafic.

Les données collectées sont ensuite sauvegardées dans le fichier pi_sensed_flows_1000.csv. Ce fichier sert d'entrée aux couches suivantes du système, notamment la couche réseau et la couche de traitement. Les métriques de collecte sont également enregistrées dans un fichier JSON appelé pi_sensing_metrics.json.

La capture affiche plusieurs mesures importantes : le nombre de lignes collectées, le nombre de flux normaux, le nombre de flux d'attaque, le temps d'exécution, l'utilisation CPU, la mémoire RAM avant et après l'exécution, ainsi que l'énergie logicielle estimée. Le temps de collecte est d'environ **129.01 ms**, avec une utilisation CPU de **25.5 %** et une mémoire supplémentaire utilisée d'environ **2.94 MB**.

L'énergie logicielle estimée est de **0.2105 J**, calculée avec la formule basée sur la durée d'exécution, la puissance nominale du Raspberry Pi et l'utilisation CPU. Cette étape montre que la collecte simulée reste légère en termes de temps, mémoire et énergie, ce qui est adapté à un environnement Edge

Tableau 2 : Résultats de la couche Sensing Layer sur Raspberry Pi

Métrique	Valeur
Flux collectes	1000
Flux benign	500
Flux attack	500
Temps Raspberry	129.01 ms
CPU	25.5 %
RAM utilisée	2.94 Mo
Energie logicielle	0.210547 J

Ce tableau résume les mesures obtenues lors de l'exécution de la couche Sensing Layer sur Raspberry Pi. Le système a collecté 1000 flux réseau, répartis de manière équilibrée entre trafic normal et trafic d'attaque. Cette collecte simulée a été réalisée en 129.01 ms, avec une utilisation CPU de 25.5 % et une consommation mémoire limitée à 2.94 Mo. L'énergie logicielle estimée pour cette étape est de 0.210547 J, ce qui montre que la phase de collecte reste peu

coûteuse par rapport aux étapes plus lourdes du pipeline, comme le chargement du modèle. Ces résultats confirment que la simulation de collecte peut être exécutée efficacement sur un dispositif Edge.

4.4 Network Layer sur Raspberry Pi :

Figure 22 : Exécution de la couche Network Layer sur Raspberry Pi

```
(iot_env) saad@IP:~/iot_project $ python pi_network_layer.py
Raspberry Pi - Network Layer
Input: outputs_iomt/pi_run/sensing_layer/pi_sensed_flows_1000.csv
label
DoS UDP    500
benign     500
Name: count, dtype: int64
          strategy  rows_sent    time_ms  energy_joules_software
0     S1_immediate       1000  21.308653                0.035458
1          S2_batch         20   8.088402                0.012941
2  S3_anomaly_only        500  10.650178                0.017040
```

Cette capture présente l'exécution de la couche **Network Layer** sur le Raspberry Pi. Cette couche a pour rôle de simuler la transmission des données collectées par la couche Sensing et de comparer plusieurs stratégies d'envoi. Le fichier utilisé contient 1000 flux réseau IoMT, répartis entre 500 flux normaux (*benign*) et 500 flux d'attaque (DoS UDP).

Trois stratégies de transmission sont évaluées. La première, S1_immediate, consiste à transmettre chaque flux immédiatement. Elle envoie donc 1000 messages, ce qui garantit une transmission complète et rapide des données, mais augmente le nombre de communications et donc le coût énergétique. La deuxième stratégie, S2_batch, regroupe les flux par lots avant transmission. Dans notre cas, elle réduit le nombre de messages à 20, ce qui diminue fortement la charge réseau et la consommation énergétique. La troisième stratégie, S3_anomaly_only, transmet uniquement les flux considérés comme suspects ou malveillants. Elle envoie donc 500 messages, correspondant aux flux d'attaque.

Les résultats montrent que S2_batch est la stratégie la plus économique, avec une énergie logicielle estimée à 0.012941 J. La stratégie S3_anomaly_only consomme légèrement plus que S2, mais elle reste plus efficace que S1 tout en étant plus adaptée à un contexte de sécurité, car elle transmet uniquement les événements suspects. La stratégie S1, bien qu'elle soit simple et complète, est la plus coûteuse en nombre de messages et en énergie.

Cette comparaison montre que le choix de la stratégie dépend du besoin du système. Pour une surveillance continue avec économie d'énergie, S2 est plus adaptée. Pour un système d'alerte orienté sécurité, S3 représente un bon compromis entre réduction du trafic et transmission des anomalies importantes.

Tableau 3 : Comparaison des stratégies de transmission réseau

Stratégie	Messages	Temps (ms)	CPU (%)	Energie (J)
S1_immediate	1000	16.14	50.0	0.0516
S2_batch	20	9.64	8.3	0.0051
S3_anomaly_only	500	13.61	15.4	0.0134

Ce tableau synthétise les résultats obtenus lors de l'exécution de la couche **Network Layer** sur Raspberry Pi. Il reprend les principales métriques affichées dans la capture précédente, à savoir le nombre de messages transmis, le temps d'exécution, l'utilisation CPU et l'énergie logicielle estimée pour chaque stratégie.

On remarque que la stratégie S1_immediate transmet tous les flux individuellement, ce qui entraîne le plus grand nombre de messages envoyés et la consommation énergétique la plus élevée. La stratégie S2_batch regroupe les données en lots, ce qui réduit fortement le nombre de transmissions et permet d'obtenir la consommation énergétique la plus faible. Enfin, la stratégie S3_anomaly_only transmet uniquement les flux suspects, ce qui constitue un bon compromis entre réduction du trafic réseau et détection des anomalies.

4.5 Data Processing Layer sur Raspberry Pi :

Figure 23 : Exécution de la couche Data Processing Layer sur Raspberry Pi

```
(iot_env) saad@IP:~/iot_project $ python pi_processing_inference.py
Raspberry Pi - Data Processing Layer
/usr/lib/python3/dist-packages/sklearn/base.py:376: InconsistentVersionWarning: Trying to unpickle estimator SimpleImputer from version 1.6.1 when using ver
sion 1.4.2. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
  warnings.warn(
/usr/lib/python3/dist-packages/sklearn/base.py:376: InconsistentVersionWarning: Trying to unpickle estimator DecisionTreeClassifier from version 1.6.1 when
using version 1.4.2. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
  warnings.warn(
/usr/lib/python3/dist-packages/sklearn/base.py:376: InconsistentVersionWarning: Trying to unpickle estimator Pipeline from version 1.6.1 when using version
1.4.2. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
  warnings.warn(
                          step      time_ms  ...  ram_used_mb  energy_joules
0                model_loading  1370.208855  ...    52.859375       2.253720
1     preprocessing_100_samples    42.545365  ...     1.839844       0.063988
2              single_inference     8.529473  ...     0.140625       0.018178
3           batch_inference_100     5.234294  ...     0.000000       0.000000
4  json_writing_100_predictions     5.624902  ...     0.015625       0.018000

[5 rows x 5 columns]
Predictions: outputs_iomt/pi_run/processing_layer/predictions_standardized.jsonl
```

Cette capture présente l'exécution de la couche **Data Processing Layer** sur Raspberry Pi. Le script pi_processing_inference.py charge le modèle de détection sérialisé, prépare les données d'entrée, effectue les prédictions et génère un fichier JSON standardisé contenant les résultats d'inférence.

Au début de l'exécution, des avertissements liés à la version de scikit-learn apparaissent. Ils indiquent que le modèle a été entraîné avec une version différente de celle utilisée sur le Raspberry Pi pour l'inférence. Ces avertissements n'empêchent pas l'exécution du programme, mais ils montrent l'importance d'utiliser des versions cohérentes des bibliothèques entre l'environnement d'entraînement et l'environnement de déploiement.

La sortie affiche ensuite les mesures détaillées des principales étapes du pipeline. On observe que le chargement du modèle est l'étape la plus coûteuse, avec un temps d'environ 1370 ms, une utilisation mémoire d'environ 52.86 Mo et une énergie estimée à 2.2537 J. Les autres étapes, comme le preprocessing, l'inférence unitaire, l'inférence en batch et l'écriture JSON, sont beaucoup plus rapides et consomment nettement moins de ressources.

Enfin, le script génère un fichier de sortie :

outputs_iomt/pi_run/processing_layer/predictions_standardized.jsonl

Ce fichier contient les prédictions au format JSON standardisé demandé, avec les champs tels que timestamp, prediction, label, confidence, model_used, energy_joules, strategy et layer. Cette étape valide donc l'exécution locale du modèle et la production des résultats exploitables par le dashboard.

Tableau 4 : Analyse des performances du pipeline d'inférence

Etape	Temps (ms)	CPU (%)	RAM (Mo)	Energie (J)
model_loading	1370.87	25.5	52.86	2.2373
Preprocessing_100_samples	43.69	23.5	1.84	0.0657
single_inference	8.68	25.0	0.14	0.0139
batch_inference_100_x100	4.72	25.0	0.004	0.0076
json_writing_100_predictions	6.23	33.3	0.02	0.0133

Ce tableau synthétise les mesures obtenues lors de l'exécution de la couche **Data Processing Layer** sur Raspberry Pi. Il reprend les principales métriques affichées dans la capture précédente, notamment le temps d'exécution, l'utilisation CPU, la mémoire RAM consommée et l'énergie logicielle estimée pour chaque étape du pipeline.

On remarque que l'étape model_loading est la plus coûteuse. Elle nécessite environ 1370.87 ms, utilise 52.86 Mo de RAM et consomme environ 2.2373 J. Cela s'explique par le chargement du modèle sérialisé et son initialisation en mémoire avant l'inférence.

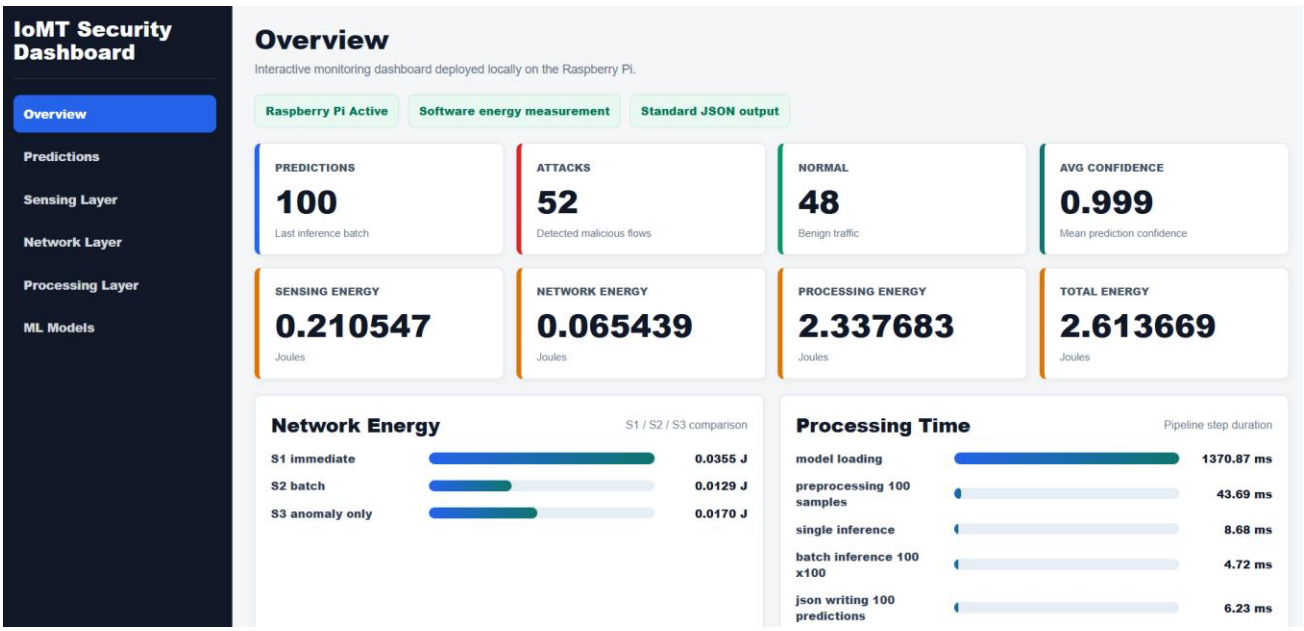
4.6 Dashboard :

Figure 24 :Lancement du dashboard Flask

```
(iot_env) saad@IP:~/iot_project $ cd ~/iot_project
source ~/iot_env/bin/activate
python app_dashboard.py
 * Serving Flask app 'app_dashboard'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.11.114:5000
Press CTRL+C to quit
192.168.11.103 - - [17/Apr/2026 00:01:10] "GET / HTTP/1.1" 200 -
```

Cette capture montre le lancement du dashboard Flask app_dashboard.py sur Raspberry Pi depuis le dossier ~/iot_project.Le serveur démarre correctement et devient accessible sur le réseau local via l'adresse http://192.168.11.114:5000.
Le code HTTP 200 confirme que la page du dashboard a été ouverte avec succès dans le navigateur.

Figure 25 : Vue générale du dashboard IoMT

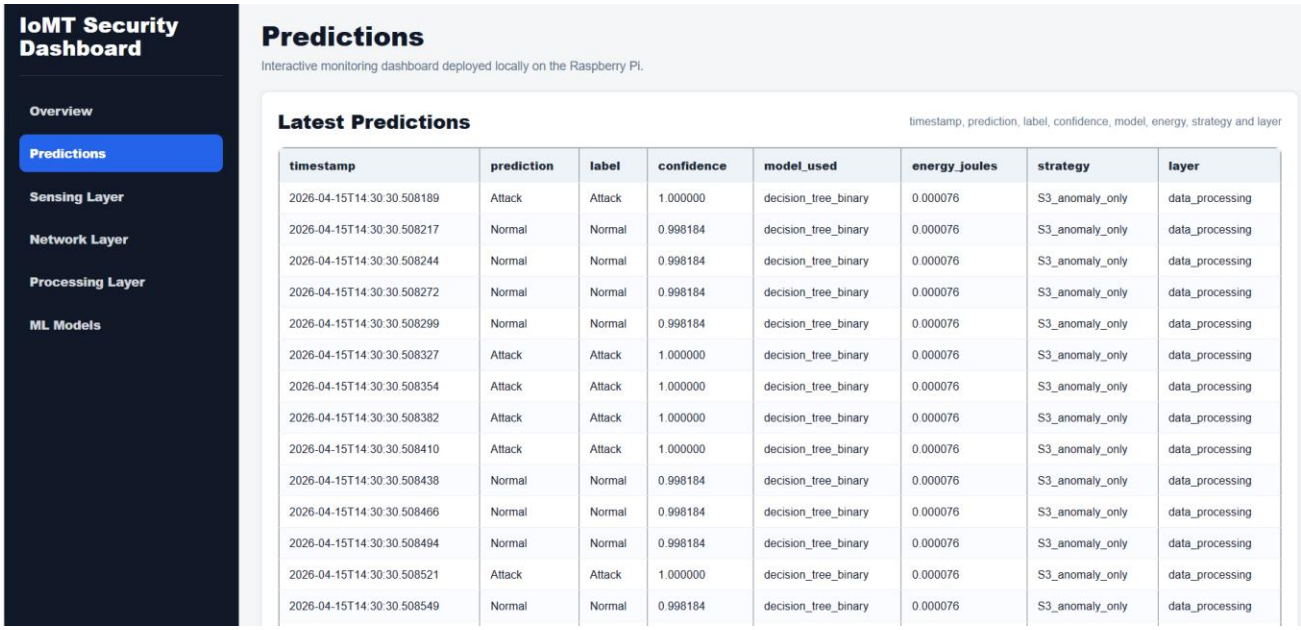


Cette capture présente la page principale du dashboard IoMT déployé sur Raspberry Pi. La barre latérale permet de naviguer entre les différentes parties du système : prédictions, Sensing Layer, Network Layer, Processing Layer et comparaison des modèles ML.

La vue Overview regroupe les indicateurs essentiels du système. Elle affiche le nombre total de prédictions réalisées, le nombre de flux détectés comme attaques, le nombre de flux normaux, ainsi que la confiance moyenne du modèle. Dans cette exécution, le système a traité 100 prédictions, avec 52 flux classés comme attaques et 48 flux classés comme normaux.

La partie inférieure résume les mesures de performance et d'énergie. Elle montre la consommation énergétique estimée pour les couches Sensing, Network et Processing, ainsi qu'une comparaison rapide des stratégies de transmission et du temps d'exécution des étapes du pipeline. Cette vue permet donc d'avoir une vision globale du comportement du système et de vérifier rapidement son fonctionnement sur Raspberry Pi.

Figure 26 : Affichage des prédictions générées par le modèle



timestamp	prediction	label	confidence	model_used	energy_joules	strategy	layer
2026-04-15T14:30:30.508189	Attack	Attack	1.000000	decision_tree_binary	0.000076	S3_anomaly_only	data_processing
2026-04-15T14:30:30.508217	Normal	Normal	0.998184	decision_tree_binary	0.000076	S3_anomaly_only	data_processing
2026-04-15T14:30:30.508244	Normal	Normal	0.998184	decision_tree_binary	0.000076	S3_anomaly_only	data_processing
2026-04-15T14:30:30.508272	Normal	Normal	0.998184	decision_tree_binary	0.000076	S3_anomaly_only	data_processing
2026-04-15T14:30:30.508299	Normal	Normal	0.998184	decision_tree_binary	0.000076	S3_anomaly_only	data_processing
2026-04-15T14:30:30.508327	Attack	Attack	1.000000	decision_tree_binary	0.000076	S3_anomaly_only	data_processing
2026-04-15T14:30:30.508354	Attack	Attack	1.000000	decision_tree_binary	0.000076	S3_anomaly_only	data_processing
2026-04-15T14:30:30.508382	Attack	Attack	1.000000	decision_tree_binary	0.000076	S3_anomaly_only	data_processing
2026-04-15T14:30:30.508410	Attack	Attack	1.000000	decision_tree_binary	0.000076	S3_anomaly_only	data_processing
2026-04-15T14:30:30.508438	Normal	Normal	0.998184	decision_tree_binary	0.000076	S3_anomaly_only	data_processing
2026-04-15T14:30:30.508466	Normal	Normal	0.998184	decision_tree_binary	0.000076	S3_anomaly_only	data_processing
2026-04-15T14:30:30.508494	Normal	Normal	0.998184	decision_tree_binary	0.000076	S3_anomaly_only	data_processing
2026-04-15T14:30:30.508521	Attack	Attack	1.000000	decision_tree_binary	0.000076	S3_anomaly_only	data_processing
2026-04-15T14:30:30.508549	Normal	Normal	0.998184	decision_tree_binary	0.000076	S3_anomaly_only	data_processing

Cette figure présente l'onglet Predictions du dashboard déployé sur Raspberry Pi. Cette partie permet de visualiser les dernières prédictions produites par le modèle de détection après l'analyse des flux réseau IoMT.

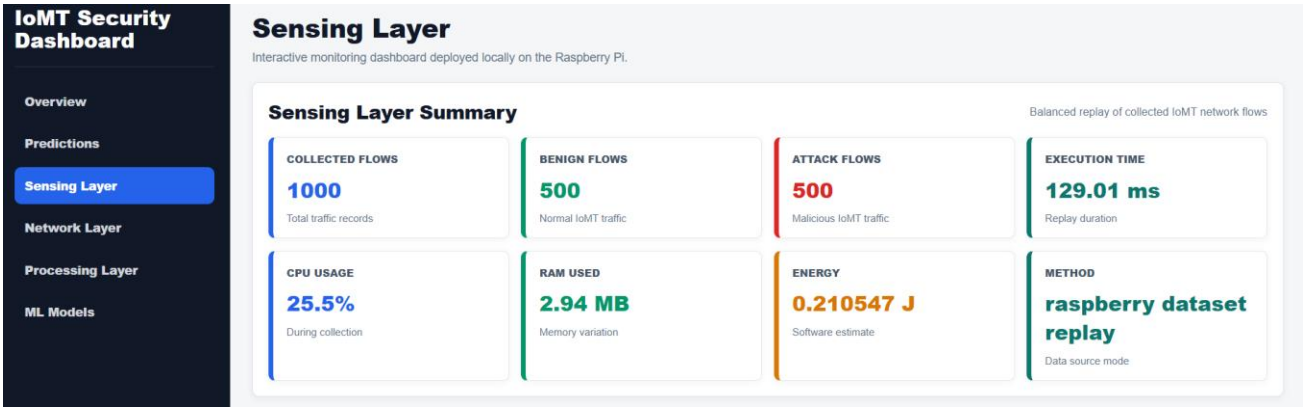
Chaque ligne du tableau correspond à un flux réseau traité par le système. La colonne timestamp indique le moment où la prédiction a été générée. La colonne prediction représente la décision du modèle, par exemple Attack pour un flux détecté

comme malveillant ou Normal pour un flux considéré comme bénin. La colonne label correspond à la classe réelle du flux, ce qui permet de comparer la prédiction du modèle avec la vérité terrain. La colonne confidence indique le niveau de confiance associé à la prédiction. Dans cette capture, les valeurs sont très élevées, proches de 1, ce qui montre que le modèle produit des décisions stables pour les flux affichés. La colonne model_used précise le modèle utilisé pour l'inférence, ici decision_tree_binary, retenu pour sa bonne performance en détection rapide Normal/Attack.

Les colonnes energy_joules, strategy et layer permettent d'assurer le suivi technique de chaque prédiction. energy_joules indique l'énergie logicielle estimée pour l'inférence, strategy précise la stratégie de transmission utilisée, et layer montre que ces résultats proviennent de la couche de traitement des données.

Cette vue est importante car elle vérifie que le système respecte le format JSON standardisé demandé dans le projet. Elle permet aussi de suivre les résultats d'inférence de manière claire et exploitable depuis l'interface web locale.

Figure 27 : Résumé de la couche Sensing Layer



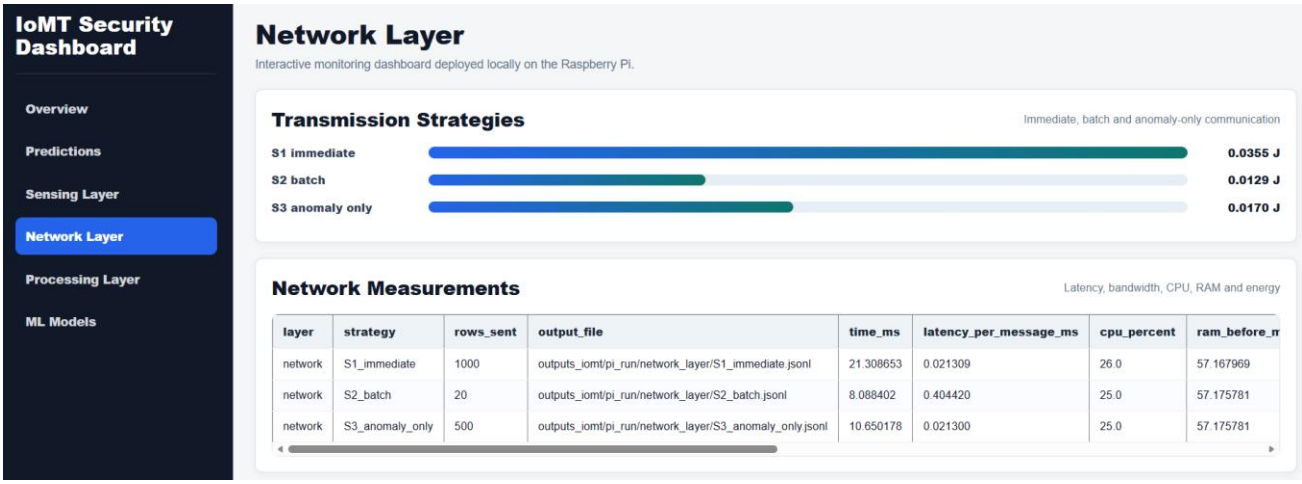
Cette capture présente l'onglet Sensing Layer du dashboard. Cette couche correspond à la phase de collecte des flux réseau IoMT utilisés par le système. Dans ce projet, la collecte est simulée à partir du dataset afin de reproduire un trafic réseau hospitalier contenant à la fois des flux normaux et des flux d'attaque.

La vue montre que 1000 flux ont été collectés, répartis de manière équilibrée entre 500 flux bénins et 500 flux d'attaque. Cette répartition permet de tester le comportement du pipeline sur

les deux types de trafic et de vérifier que les couches suivantes reçoivent des données représentatives.

Les mesures affichées indiquent également les ressources consommées pendant cette étape : un temps d'exécution de 129.01 ms, une utilisation CPU de 25.5 %, une consommation mémoire de 2.94 MB et une énergie logicielle estimée à 0.210547 J. Ces résultats montrent que la phase de collecte simulée reste légère et peut être exécutée efficacement dans un environnement Edge.

Figure 28 : Résumé de la couche Sensing Layer



layer	strategy	rows_sent	output_file	time_ms	latency_per_message_ms	cpu_percent	ram_before_m
network	S1_immediate	1000	outputs_iomt/pi_run/network_layer/S1_immediate.jsonl	21.308653	0.021309	26.0	57.167969
network	S2_batch	20	outputs_iomt/pi_run/network_layer/S2_batch.jsonl	8.088402	0.404420	25.0	57.175781
network	S3_anomaly_only	500	outputs_iomt/pi_run/network_layer/S3_anomaly_only.jsonl	10.650178	0.021300	25.0	57.175781

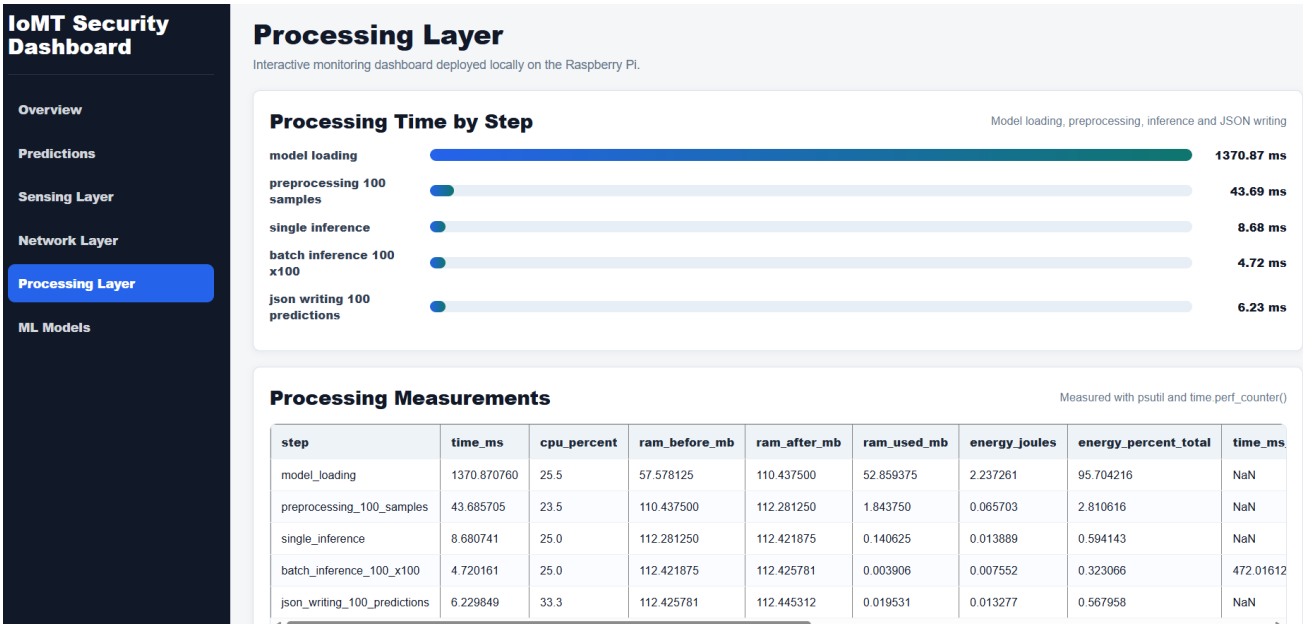
Cette capture présente l'onglet Network Layer du dashboard. Cette couche simule la transmission des données collectées selon trois stratégies :

S1_immediate, S2_batch et S3_anomaly_only. L'objectif est de comparer leur impact sur le nombre de messages envoyés, le temps d'exécution, la latence, l'utilisation des ressources et l'énergie logicielle estimée.

La stratégie S1_immediate envoie chaque flux individuellement, ce qui génère 1000 messages et entraîne la consommation énergétique la plus élevée. La stratégie S2_batch regroupe les flux en lots, ce qui réduit fortement le nombre de transmissions à 20 messages et donne la plus faible consommation énergétique. La stratégie S3_anomaly_only transmet uniquement les flux détectés comme suspects, soit 500 messages, ce qui représente un compromis entre réduction du trafic réseau et conservation des informations importantes.

Le tableau inférieur détaille les mesures obtenues pour chaque stratégie, notamment le temps d'exécution, la latence par message, le CPU, la RAM et les fichiers de sortie générés. Ces résultats montrent que la transmission par batch est la solution la plus efficace énergétiquement, tandis que la transmission conditionnelle aux anomalies peut être intéressante lorsque l'objectif est de limiter le trafic sans perdre les alertes de sécurité.

Figure 29 : Mesures de performance de la couche Data Processing Layer



step	time_ms	cpu_percent	ram_before_mb	ram_after_mb	ram_used_mb	energy_joules	energy_percent_total	time_ms
model_loading	1370.870760	25.5	57.578125	110.437500	52.859375	2.237261	95.704216	NaN
preprocessing_100_samples	43.685705	23.5	110.437500	112.281250	1.843750	0.065703	2.810616	NaN
single_inference	8.680741	25.0	112.281250	112.421875	0.140625	0.013889	0.594143	NaN
batch_inference_100_x100	4.720161	25.0	112.421875	112.425781	0.003906	0.007552	0.323066	472.01612
json_writing_100_predictions	6.229849	33.3	112.425781	112.445312	0.019531	0.013277	0.567958	NaN

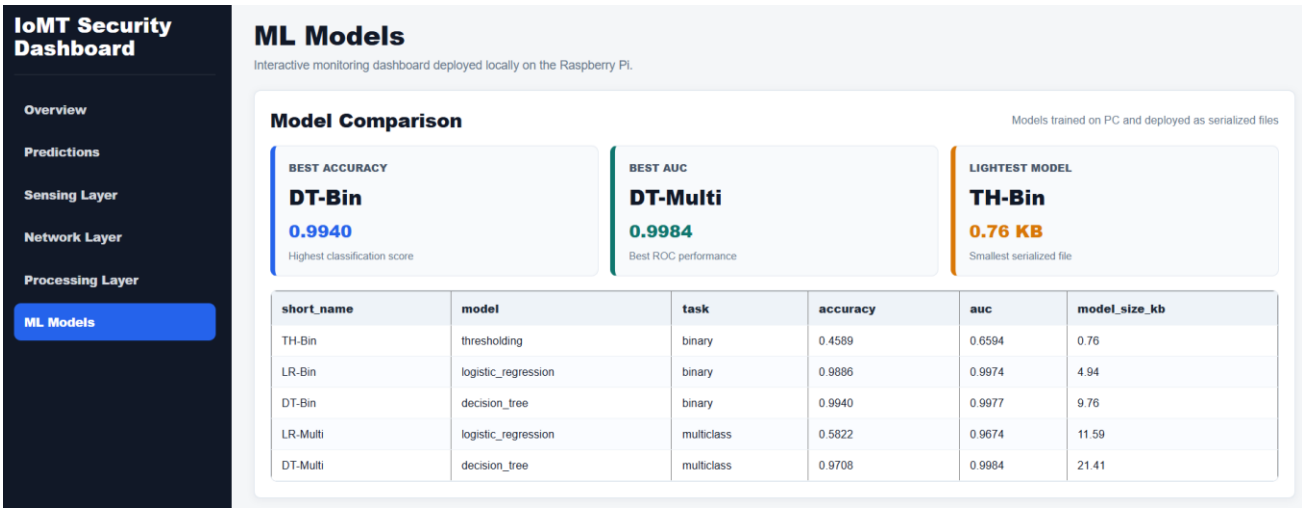
Cette capture présente l'onglet Processing Layer du dashboard. Cette couche correspond au traitement intelligent des données, incluant le chargement du modèle, le preprocessing des échantillons, l'inférence et l'écriture des prédictions au format JSON standardisé.

Le graphique supérieur compare le temps d'exécution des différentes étapes du pipeline. On observe que le chargement du modèle est l'étape la plus coûteuse, avec environ 1370.87 ms. Les autres étapes sont beaucoup plus rapides : le preprocessing de 100 échantillons prend environ 43.69 ms, l'inférence unitaire 8.68 ms, l'inférence en batch 4.72 ms et l'écriture JSON 6.23 ms.

Le tableau inférieur détaille les mesures obtenues pour chaque étape : temps d'exécution, CPU, RAM utilisée, énergie logicielle estimée et pourcentage d'énergie consommée. Les résultats montrent que la majorité du coût provient du chargement du modèle, tandis que l'inférence elle-

même reste légère. Cela justifie le choix de charger le modèle une seule fois au démarrage, puis de le réutiliser pour les prédictions suivantes.

Figure 30 : Comparaison des modèles de Machine Learning déployés



short_name	model	task	accuracy	auc	model_size_kb
TH-Bin	thresholding	binary	0.4589	0.6594	0.76
LR-Bin	logistic_regression	binary	0.9886	0.9974	4.94
DT-Bin	decision_tree	binary	0.9940	0.9977	9.76
LR-Multi	logistic_regression	multiclass	0.5822	0.9674	11.59
DT-Multi	decision_tree	multiclass	0.9708	0.9984	21.41

Cette capture présente l'onglet ML Models du dashboard. Il résume les performances des modèles entraînés sur PC puis exportés sous forme de fichiers sérialisés pour être utilisés sur Raspberry Pi. Les modèles comparés sont le Thresholding, la Logistic Regression et le Decision Tree, en classification binaire et multi-classe.

Les cartes supérieures mettent en évidence les résultats principaux. Le modèle DT-Bin obtient la meilleure accuracy avec une valeur de 0.9940, ce qui en fait le modèle le plus performant en classification binaire. Le modèle DT-Multi obtient le meilleur AUC avec une valeur de 0.9984, montrant une très bonne capacité de séparation entre les classes en configuration multi-classe. Le modèle TH-Bin est le plus léger, avec une taille de seulement 0.76 KB, mais ses performances restent beaucoup plus faibles.

Le tableau inférieur détaille pour chaque modèle la tâche réalisée, l'accuracy, l'AUC et la taille du fichier sérialisé. Ces résultats montrent que le Decision Tree offre le meilleur compromis entre performance, taille raisonnable du modèle et adaptation à une exécution sur dispositif Edge. C'est pourquoi il a été retenu comme modèle principal pour l'inférence.

4.7 Analyse finale :

Les résultats obtenus montrent que le modèle Decision Tree binaire est le plus adapté pour une détection rapide du trafic réseau IoMT. Il permet de distinguer efficacement les flux Normal et Attack, avec une accuracy élevée et un temps d'inférence faible. Cette configuration est particulièrement intéressante pour un système Edge, car elle fournit une décision simple, rapide et directement exploitable pour déclencher une alerte.

Le Decision Tree multi-classe apporte une analyse plus détaillée, car il ne se limite pas à détecter la présence d'une attaque, mais cherche aussi à identifier son type. Cette approche est utile pour l'analyse de sécurité, car elle permet de mieux comprendre la nature des menaces détectées. Cependant, elle reste légèrement plus coûteuse que la classification binaire et certaines classes rares sont plus difficiles à reconnaître correctement en raison de leur faible représentation dans le dataset.

L'optimisation locale des modèles avant déploiement montre également l'intérêt d'une approche Edge/TinyML. La conversion des modèles compatibles vers le format ONNX permet de réduire significativement leur taille, avec des gains allant d'environ 43 % à 63 % par rapport aux fichiers joblib. Cette réduction facilite le transfert des modèles vers le Raspberry Pi et diminue l'espace de stockage nécessaire, tout en conservant une architecture adaptée à l'inférence.

Concernant la couche réseau, la stratégie S2_batch est la plus économique en énergie, car elle regroupe les données avant leur transmission. Cela réduit fortement le nombre de messages envoyés et limite donc le coût de communication. En revanche, la stratégie S3_anomaly_only est plus adaptée à un système d'alerte, car elle transmet uniquement les flux suspects. Elle représente donc un bon compromis entre réduction du trafic réseau et conservation des informations importantes pour la détection d'intrusions.

Enfin, les mesures réalisées sur Raspberry Pi confirment que le système peut fonctionner dans un environnement Edge avec des ressources limitées. Le chargement du modèle reste l'étape la plus coûteuse, tandis que l'inférence et l'écriture JSON sont rapides. Il est donc préférable de charger le modèle une seule fois au démarrage, puis de le réutiliser pour traiter les flux suivants.

Globalement, l'architecture proposée répond aux objectifs du projet : détecter les anomalies réseau, comparer les modèles de Machine Learning, optimiser les modèles avant déploiement, comparer les stratégies de transmission, mesurer les ressources consommées et visualiser les résultats dans un dashboard local. Le système obtenu montre ainsi un bon compromis entre performance de détection, légèreté du modèle, consommation de ressources et exploitabilité dans un environnement IoMT hospitalier.

4.8 Conclusion :

Ce chapitre a présenté le déploiement du système sur Raspberry Pi, l'optimisation locale des modèles avant déploiement, ainsi que l'exécution des différentes couches du pipeline. Les résultats obtenus montrent que le système peut réaliser l'inférence, mesurer les ressources consommées et afficher les prédictions dans un dashboard local. Cette expérimentation confirme la faisabilité d'une approche Edge pour le monitoring de la sécurité réseau IoMT.

Conclusion générale :

Ce projet a permis de concevoir et de mettre en œuvre un système IoT de monitoring de la sécurité réseau dans un environnement hospitalier IoMT. L'objectif principal était de détecter les anomalies et intrusions dans le trafic réseau des dispositifs médicaux connectés, tout en tenant compte des contraintes d'un dispositif Edge tel que le Raspberry Pi.

L'architecture proposée repose sur quatre couches complémentaires : la couche de collecte des flux réseau, la couche de transmission, la couche de traitement intelligent et la couche applicative dédiée à la visualisation. Cette organisation permet de couvrir l'ensemble du pipeline, depuis la récupération des données jusqu'à l'affichage des prédictions, des métriques de performance et des mesures énergétiques dans un dashboard local.

Les modèles de Machine Learning ont été entraînés sur PC à partir d'un échantillon stratifié du dataset CICIoMT2024. Cette stratégie a permis de réduire le volume de données traité tout en conservant la distribution des classes. Les modèles ont ensuite été exportés, optimisés localement et déployés sur Raspberry Pi pour la phase d'inférence. La conversion ONNX a permis de réduire la taille des modèles compatibles, ce qui renforce l'adaptation du système à un environnement Edge.

Les résultats montrent que le `Decision Tree` binaire offre le meilleur compromis pour une détection rapide `Normal/Attack`, avec une accuracy élevée, une taille raisonnable et un temps d'inférence faible. Le `Decision Tree` multi-classe permet quant à lui une analyse plus détaillée en identifiant le type d'attaque, même si certaines classes rares restent plus difficiles à reconnaître correctement.

Les mesures réalisées sur Raspberry Pi confirment l'importance d'analyser les ressources consommées par chaque étape du pipeline. Le chargement du modèle représente l'étape la plus coûteuse en temps, en mémoire et en énergie, tandis que l'inférence et l'écriture JSON restent légères. Du côté réseau, la stratégie `S2_batch` réduit fortement le coût énergétique grâce au regroupement des messages, alors que `S3_anomaly_only` constitue une solution intéressante pour un système d'alerte basé sur la transmission des flux suspects uniquement.

Globalement, le système développé répond aux objectifs du projet. Il permet de détecter les anomalies réseau, de comparer plusieurs modèles, d'optimiser les fichiers avant déploiement, de mesurer les ressources consommées et de visualiser les résultats à travers un dashboard local. Ce travail montre l'intérêt d'une approche Edge Computing pour renforcer la sécurité des réseaux IoMT hospitaliers.